





**Technical Implementation Guide:  
Modified Cooperative Cruise Controller  
(MYCC) for PLEXE Truck Platooning**  
Incorporating SUMO, CityFlow, Deep Reinforcement Learning, Grid  
Generation, and Advanced Traffic Analysis with TRAFI

Research Team

March 26, 2026

# Abstract

This technical guide provides a comprehensive exposition of the design, implementation, and validation of the Modified Cooperative Cruise Controller (MYCC) within the PLEXE framework. The MYCC enhances truck platooning stability during complex manoeuvres such as highway on-ramp merges. The implementation integrates two complementary microscopic traffic simulators—SUMO and CityFlow—and leverages deep reinforcement learning for coordinated traffic signal control. Detailed source code modifications, build procedures, simulation configuration, and data analysis scripts are presented. Extensive validation through braking and sinusoidal tests demonstrates significant improvements in gap maintenance, fuel efficiency, and emissions reduction compared to baseline controllers (IDM, ACC, CACC). The guide also explores advanced topics, including multi-agent reinforcement learning (MARL) for network-wide signal coordination, the internal architecture of CityFlow (segment lists, cross points), grid generation tools for creating synthetic road networks, and the TRAFI traffic analysis suite for generating hard scenarios (peak congestion, emergency traffic) and evaluating system performance. A comprehensive bibliography and glossary of terms are provided.

# Acknowledgements

The authors gratefully acknowledge the support of the German Aerospace Center (DLR) for developing SUMO, the creators of PLEXE and Veins, the CityFlow development team, and the contributors to the open-source simulation ecosystem.

# Revision History

| Version | Date       | Changes                                                                                             |
|---------|------------|-----------------------------------------------------------------------------------------------------|
| 1.0     | 2025-12-26 | Initial release                                                                                     |
| 1.1     | 2026-01-17 | Added quickstart guide, string stability analysis, and troubleshooting                              |
| 1.2     | 2026-02-16 | Incorporated abstract, glossary, improved cross-referencing, and refined bibliography               |
| 1.3     | 2026-02-17 | Expanded CityFlow internals (segment lists, cross points), added grid generation and TRAFI sections |
| 1.4     | 2026-03-20 | Integrated DeepRL Configuration, Experiment Management, and MoST Scenario Documentation             |

# Quickstart Guide

**Objective:** Execute a MYCC-controlled platoon simulation within ten minutes.

## 1. Clone the required repositories:

```
1 git clone --recursive https://github.com/eclipse/sumo.git
2 git clone https://github.com/msegata/plexo.git
3 git clone https://github.com/sommer/veins.git
4 git clone https://github.com/cityflow/CityFlow.git
5
```

## 2. Apply the provided patches (see Appendix A):

```
1 cd sumo
2 patch -p1 < patches/sumo_mycc.patch
3 cd plexo
4 patch -p1 < patches/plexo_mycc.patch
5
```

## 3. Build the frameworks (detailed instructions in Chapter 10):

```
1 cd sumo && mkdir build && cd build && cmake .. && make -j4
2 cd veins && ./configure && make
3 cd plexo && ./configure && make
4 cd CityFlow && mkdir build && cd build && cmake .. && make
5
```

## 4. Execute a test simulation:

```
1 cd plexo/examples/platooning
2 plexo_run -u Cmdenv -c BrakingNoGui -r 0
3
```

## 5. Visualise the results using the R scripts provided in Chapter 12.

All code and patches are available at: <https://github.com/mycc-plexo>

# Contents

|                                                                           |            |
|---------------------------------------------------------------------------|------------|
| <b>Abstract</b>                                                           | <b>ii</b>  |
| <b>Acknowledgements</b>                                                   | <b>iii</b> |
| <b>Revision History</b>                                                   | <b>iv</b>  |
| <b>Quickstart Guide</b>                                                   | <b>v</b>   |
| <b>List of Abbreviations</b>                                              | <b>ix</b>  |
| <br>                                                                      |            |
| <b>I Foundations</b>                                                      | <b>1</b>   |
| <br>                                                                      |            |
| <b>1 Introduction</b>                                                     | <b>2</b>   |
| 1.1 Overview . . . . .                                                    | 2          |
| 1.2 Controller Formulation . . . . .                                      | 2          |
| 1.3 Document Structure . . . . .                                          | 3          |
| <br>                                                                      |            |
| <b>2 Microscopic Traffic Simulation Frameworks</b>                        | <b>4</b>   |
| 2.1 Introduction to Traffic Simulation . . . . .                          | 4          |
| 2.2 SUMO (Simulation of Urban MObility) . . . . .                         | 4          |
| 2.2.1 Architecture and Design Philosophy . . . . .                        | 4          |
| 2.2.2 Network Representation . . . . .                                    | 5          |
| 2.2.3 Car-Following Models . . . . .                                      | 5          |
| 2.2.4 Integration Schemes . . . . .                                       | 6          |
| 2.2.5 Action Point Mechanism . . . . .                                    | 6          |
| 2.2.6 Sublane Model . . . . .                                             | 6          |
| 2.3 CityFlow: High-Performance Traffic Simulation . . . . .               | 6          |
| 2.3.1 Motivation and Design Objectives . . . . .                          | 6          |
| 2.3.2 Optimised Data Structures: Segment Lists and Cross Points . . . . . | 7          |
| 2.3.3 Car-Following Implementation . . . . .                              | 8          |
| 2.3.4 Injecting Custom Controllers (MYCC Integration) . . . . .           | 8          |
| 2.3.5 Python Interface for RL and Control . . . . .                       | 9          |
| 2.3.6 Performance Comparison . . . . .                                    | 10         |
| 2.4 TraCI: The Traffic Control Interface . . . . .                        | 10         |
| 2.4.1 Protocol Overview . . . . .                                         | 10         |
| 2.4.2 Message Structure . . . . .                                         | 10         |
| 2.4.3 Performance Considerations . . . . .                                | 11         |
| 2.4.4 TraCI Communication Flow . . . . .                                  | 11         |

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| <b>3</b>  | <b>Deep Reinforcement Learning for Traffic Control</b> | <b>12</b> |
| 3.1       | Reinforcement Learning Fundamentals                    | 12        |
| 3.1.1     | Markov Decision Process                                | 12        |
| 3.1.2     | Deep Q-Learning                                        | 12        |
| 3.1.3     | Policy Gradient Methods                                | 13        |
| 3.2       | Multi-Agent Reinforcement Learning (MARL)              | 13        |
| 3.2.1     | The MARL Challenge                                     | 13        |
| 3.2.2     | Independent versus Joint Learning                      | 13        |
| 3.2.3     | Coordination Mechanisms                                | 14        |
| 3.3       | MARL for Traffic Signal Control                        | 14        |
| 3.3.1     | Problem Formulation                                    | 14        |
| 3.3.2     | Novel Reward Strategies                                | 14        |
| 3.3.3     | State-of-the-Art MARL Methods                          | 15        |
| 3.3.4     | Reward Shaping Challenges                              | 16        |
| 3.4       | Integrating Platooning with Traffic Signal Control     | 16        |
| 3.4.1     | The Coordination Opportunity                           | 16        |
| 3.4.2     | Hierarchical Control Architecture                      | 16        |
| 3.4.3     | Communication Framework                                | 17        |
| <br>      |                                                        |           |
| <b>II</b> | <b>MYCC Controller Design</b>                          | <b>18</b> |
| <br>      |                                                        |           |
| <b>4</b>  | <b>Control Theory for Platooning</b>                   | <b>19</b> |
| 4.1       | Cooperative Adaptive Cruise Control                    | 19        |
| 4.1.1     | CACC Fundamentals                                      | 19        |
| 4.1.2     | Control Architectures                                  | 19        |
| 4.1.3     | Spacing Policies                                       | 19        |
| 4.2       | MYCC Control Law Derivation                            | 20        |
| 4.2.1     | PD Control Framework                                   | 20        |
| 4.2.2     | Parameter Selection                                    | 20        |
| 4.2.3     | Stability Analysis                                     | 20        |
| 4.2.4     | String Stability                                       | 21        |
| 4.2.5     | Parameter Sensitivity Analysis                         | 21        |
| 4.3       | Comparison with Baseline Controllers                   | 21        |
| 4.3.1     | IDM (Intelligent Driver Model)                         | 22        |
| 4.3.2     | ACC (Adaptive Cruise Control)                          | 22        |
| 4.3.3     | CACC (Cooperative ACC)                                 | 22        |
| <br>      |                                                        |           |
| <b>5</b>  | <b>Simulation Environment Configuration</b>            | <b>23</b> |
| 5.1       | Network Design                                         | 23        |
| 5.1.1     | Highway Segment Geometry                               | 23        |
| 5.1.2     | Network Creation with NETCONVERT                       | 23        |
| 5.2       | Traffic Demand Generation                              | 24        |
| 5.2.1     | Platoon Definition                                     | 24        |
| 5.2.2     | Random Traffic Generation                              | 24        |
| 5.3       | Detector Configuration                                 | 25        |
| 5.3.1     | Induction Loop Detectors                               | 25        |

|            |                                                        |           |
|------------|--------------------------------------------------------|-----------|
| <b>6</b>   | <b>MARL Integration for Multi-Intersection Control</b> | <b>26</b> |
| 6.1        | Multi-Agent Problem Formulation                        | 26        |
| 6.1.1      | Network Architecture                                   | 26        |
| 6.1.2      | State Space                                            | 26        |
| 6.1.3      | Action Space                                           | 26        |
| 6.1.4      | Reward Design                                          | 27        |
| 6.2        | SocialLight Implementation                             | 27        |
| 6.2.1      | Algorithm Overview                                     | 27        |
| 6.2.2      | Local Critic Architecture                              | 27        |
| 6.2.3      | Counterfactual Advantage                               | 27        |
| 6.3        | Integration with Platooning                            | 28        |
| 6.3.1      | Hierarchical Control Architecture                      | 28        |
| 6.3.2      | Information Flow                                       | 28        |
| 6.3.3      | On-ramp Merging Coordination                           | 28        |
| <b>III</b> | <b>Implementation</b>                                  | <b>29</b> |
| <b>7</b>   | <b>SUMO Source Modifications</b>                       | <b>30</b> |
| 7.1        | Code Architecture Overview                             | 30        |
| 7.1.1      | Core Components                                        | 30        |
| 7.1.2      | Modification Strategy                                  | 30        |
| 7.2        | Header File Modifications                              | 30        |
| 7.2.1      | CC_Const.h                                             | 30        |
| 7.2.2      | CC_VehicleVariables.h                                  | 31        |
| 7.3        | Implementation in MSCFModel_CC.cpp                     | 31        |
| 7.3.1      | Controller Factory Method                              | 31        |
| 7.3.2      | MYCC Control Law Implementation                        | 32        |
| 7.3.3      | Parameter Handling                                     | 32        |
| 7.4        | TraCI Integration                                      | 33        |
| 7.4.1      | TraCI Command Extensions                               | 33        |
| <b>8</b>   | <b>PLEXE Framework Integration</b>                     | <b>34</b> |
| 8.1        | PLEXE Architecture                                     | 34        |
| 8.1.1      | Framework Overview                                     | 34        |
| 8.1.2      | Component Structure                                    | 34        |
| 8.2        | MYCC Integration Steps                                 | 34        |
| 8.2.1      | BaseScenario Modifications                             | 34        |
| 8.2.2      | NED File Extensions                                    | 35        |
| <b>9</b>   | <b>Simulation Configuration</b>                        | <b>36</b> |
| 9.1        | OMNeT++ Configuration                                  | 36        |
| 9.1.1      | omnetpp.ini Structure                                  | 36        |
| 9.2        | SUMO Configuration                                     | 37        |
| 9.2.1      | SUMO Configuration File                                | 37        |
| 9.3        | CityFlow Configuration (Alternative)                   | 37        |
| 9.3.1      | CityFlow Roadnet File                                  | 37        |

|           |                                                                      |           |
|-----------|----------------------------------------------------------------------|-----------|
| <b>IV</b> | <b>Execution and Analysis</b>                                        | <b>39</b> |
| <b>10</b> | <b>Build and Compilation</b>                                         | <b>40</b> |
| 10.1      | SUMO Build Process . . . . .                                         | 40        |
| 10.1.1    | Prerequisites . . . . .                                              | 40        |
| 10.1.2    | Building Modified SUMO . . . . .                                     | 40        |
| 10.2      | OMNeT++ Build Process . . . . .                                      | 41        |
| 10.2.1    | OMNeT++ Installation . . . . .                                       | 41        |
| 10.3      | Docker Alternative . . . . .                                         | 41        |
| 10.3.1    | Dockerfile . . . . .                                                 | 41        |
| 10.4      | Troubleshooting Common Build Issues . . . . .                        | 42        |
| <b>11</b> | <b>Simulation Execution</b>                                          | <b>43</b> |
| 11.1      | Running Simulations . . . . .                                        | 43        |
| 11.1.1    | Basic Execution . . . . .                                            | 43        |
| 11.1.2    | Batch Processing . . . . .                                           | 43        |
| 11.2      | Data Collection . . . . .                                            | 44        |
| 11.2.1    | Output Files . . . . .                                               | 44        |
| 11.2.2    | Custom Output with TraCI . . . . .                                   | 44        |
| <b>12</b> | <b>Results Analysis</b>                                              | <b>46</b> |
| 12.1      | R Scripts for Data Processing . . . . .                              | 46        |
| 12.1.1    | Data Loading and Preprocessing . . . . .                             | 46        |
| 12.1.2    | Braking Test Analysis . . . . .                                      | 46        |
| 12.1.3    | Sample Output Plots . . . . .                                        | 47        |
| 12.2      | Performance Metrics . . . . .                                        | 47        |
| 12.2.1    | Key Performance Indicators . . . . .                                 | 47        |
| 12.2.2    | Statistical Validation . . . . .                                     | 47        |
| 12.2.3    | Performance Benchmarking . . . . .                                   | 48        |
| <b>V</b>  | <b>Advanced Topics</b>                                               | <b>49</b> |
| <b>13</b> | <b>CityFlow Grid Generation and Scenario Creation</b>                | <b>50</b> |
| 13.1      | Grid Generator Tool . . . . .                                        | 50        |
| 13.1.1    | Key Parameters . . . . .                                             | 50        |
| 13.1.2    | Generated Files . . . . .                                            | 50        |
| 13.2      | Pre-generated Grid Examples . . . . .                                | 51        |
| <b>14</b> | <b>TRAFI: Advanced Traffic Analysis and Hard Scenario Generation</b> | <b>52</b> |
| 14.1      | Key Features . . . . .                                               | 52        |
| 14.2      | Hard Scenario Generation . . . . .                                   | 52        |
| 14.2.1    | Generating Scenarios . . . . .                                       | 53        |
| 14.2.2    | Sample Output . . . . .                                              | 53        |
| 14.3      | Integration with RL Frameworks . . . . .                             | 53        |

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>15 MARL for Traffic Signal Control</b>              | <b>54</b> |
| 15.1 Multi-Agent Problem Formulation . . . . .         | 54        |
| 15.1.1 Network-Level Objectives . . . . .              | 54        |
| 15.2 SocialLight Implementation . . . . .              | 54        |
| 15.2.1 Algorithm Overview . . . . .                    | 54        |
| 15.2.2 Complete Runnable Example . . . . .             | 54        |
| <b>16 MARL Implementation in SUMO</b>                  | <b>56</b> |
| 16.1 SUMO-MARL Interface . . . . .                     | 56        |
| 16.1.1 TraCI for Multi-Agent Control . . . . .         | 56        |
| <b>17 Future Directions</b>                            | <b>57</b> |
| 17.1 MARL-CACC Integration . . . . .                   | 57        |
| 17.1.1 Unified Control Framework . . . . .             | 57        |
| 17.1.2 Multi-Objective Optimisation . . . . .          | 57        |
| 17.2 Open Challenges . . . . .                         | 57        |
| 17.2.1 Scalability . . . . .                           | 57        |
| 17.2.2 Robustness and Real-World Deployment . . . . .  | 58        |
| 17.2.3 Safety Guarantees . . . . .                     | 58        |
| <b>A Complete Code Diffs</b>                           | <b>59</b> |
| A.1 SUMO Source Changes . . . . .                      | 59        |
| A.2 PLEXE Changes . . . . .                            | 59        |
| <b>B Team Contributions</b>                            | <b>61</b> |
| B.1 Weekly Progress Summary . . . . .                  | 61        |
| B.2 Individual Contributions . . . . .                 | 62        |
| <b>C DeepRL Configuration Guide</b>                    | <b>63</b> |
| C.1 Configuration File Structure . . . . .             | 63        |
| C.1.1 Available Algorithms . . . . .                   | 63        |
| C.2 Model Configuration ([MODEL_CONFIG]) . . . . .     | 63        |
| C.3 Training Configuration ([TRAIN_CONFIG]) . . . . .  | 64        |
| C.4 Environment Configuration ([ENV_CONFIG]) . . . . . | 64        |
| C.5 Example Configuration . . . . .                    | 65        |
| <b>D Simulation Data Generation Scripts</b>            | <b>66</b> |
| D.1 Directory Structure . . . . .                      | 66        |
| D.2 Grid Generation (build_file.py) . . . . .          | 66        |
| D.2.1 Key Parameters . . . . .                         | 66        |
| D.2.2 Network Architecture . . . . .                   | 67        |
| D.3 Execution . . . . .                                | 67        |
| <b>E Experiment Management</b>                         | <b>68</b> |
| E.1 Directory Structure . . . . .                      | 68        |
| E.2 Algorithm Architectures . . . . .                  | 68        |
| E.2.1 Common Architecture . . . . .                    | 68        |
| E.3 Experiment Configurations . . . . .                | 68        |
| E.4 Analysis Commands . . . . .                        | 69        |

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>F</b> | <b>MoST Real-World Scenario Setup</b> | <b>70</b> |
| F.1      | Prerequisites . . . . .               | 70        |
| F.2      | Network Components . . . . .          | 70        |
| F.3      | Dynamic Route Generation . . . . .    | 70        |
| F.3.1    | Flow Patterns . . . . .               | 70        |
| F.4      | RL Integration Interface . . . . .    | 71        |

# List of Figures

|      |                                                              |    |
|------|--------------------------------------------------------------|----|
| 2.1  | TraCI communication flow between OMNeT++ and SUMO. . . . .   | 11 |
| 3.1  | Reinforcement learning fundamentals. . . . .                 | 12 |
| 4.1  | Bode plot of the string stability transfer function. . . . . | 21 |
| 12.1 | Sample speed plot from braking test. . . . .                 | 47 |
| 14.1 | Peak congestion analysis from TRAFI. . . . .                 | 53 |

# List of Tables

|      |                                                                       |    |
|------|-----------------------------------------------------------------------|----|
| 1.1  | MYCC Design Parameters . . . . .                                      | 2  |
| 2.1  | Performance Comparison: SUMO vs. CityFlow (steps/sec) . . . . .       | 10 |
| 4.1  | Parameter Sensitivity Analysis . . . . .                              | 21 |
| 4.2  | Comparison of Platooning Controllers . . . . .                        | 21 |
| 6.1  | Hierarchical control architecture for the integrated system . . . . . | 28 |
| 7.1  | SUMO car-following source files . . . . .                             | 30 |
| 8.1  | PLEXE core components . . . . .                                       | 34 |
| 10.1 | Common Build Issues and Solutions . . . . .                           | 42 |
| 11.1 | Simulation output files . . . . .                                     | 44 |
| 12.1 | MYCC Performance Summary . . . . .                                    | 47 |
| 12.2 | Simulation Performance Benchmark . . . . .                            | 48 |
| 13.1 | Grid Generator Arguments . . . . .                                    | 50 |
| 13.2 | Pre-generated Grid Sizes . . . . .                                    | 51 |
| B.1  | Team Contribution Summary . . . . .                                   | 61 |
| B.2  | Detailed Individual Contributions . . . . .                           | 62 |
| C.1  | Model Hyperparameters . . . . .                                       | 64 |
| C.2  | Training Schedule Parameters . . . . .                                | 64 |
| C.3  | Environment Parameters . . . . .                                      | 65 |
| D.1  | Grid Generator Parameters . . . . .                                   | 66 |
| F.1  | MoST Scenario Files . . . . .                                         | 70 |

# List of Abbreviations

|                  |                                                            |
|------------------|------------------------------------------------------------|
| <b>ACC</b>       | Adaptive Cruise Control                                    |
| <b>CACC</b>      | Cooperative Adaptive Cruise Control                        |
| <b>CTDE</b>      | Centralized Training Decentralized Execution               |
| <b>Dec-POMDP</b> | Decentralized Partially Observable Markov Decision Process |
| <b>DQN</b>       | Deep Q-Network                                             |
| <b>GAE</b>       | Generalized Advantage Estimation                           |
| <b>IA2C</b>      | Independent Actor-Critic                                   |
| <b>IDM</b>       | Intelligent Driver Model                                   |
| <b>MA2C</b>      | Multi-Agent Actor-Critic                                   |
| <b>MARL</b>      | Multi-Agent Reinforcement Learning                         |
| <b>MDP</b>       | Markov Decision Process                                    |
| <b>MoST</b>      | Monaco SUMO Traffic                                        |
| <b>MYCC</b>      | Modified Cooperative Cruise Controller                     |
| <b>PLEXE</b>     | Platooning Extension for Veins                             |
| <b>RL</b>        | Reinforcement Learning                                     |
| <b>SUMO</b>      | Simulation of Urban MObility                               |
| <b>TraCI</b>     | Traffic Control Interface                                  |
| <b>V2I</b>       | Vehicle-to-Infrastructure                                  |
| <b>V2V</b>       | Vehicle-to-Vehicle                                         |
| <b>V2X</b>       | Vehicle-to-Everything                                      |
| <b>TRAFFI</b>    | Traffic Flow Analyzer and Simulator                        |

# Part I

## Foundations

# Chapter 1

## Introduction

### 1.1 Overview

This document delineates the complete implementation details for the Modified Cooperative Cruise Controller (MYCC) within the PLEXE framework. The MYCC extends the cooperative cruise control capabilities of SUMO to facilitate stable truck platooning, particularly during complex manoeuvres such as highway on-ramp merges. The implementation leverages microscopic traffic simulation principles from both SUMO and CityFlow, thereby establishing a robust testing environment for cooperative adaptive cruise control strategies.

### 1.2 Controller Formulation

The MYCC law combines distance and velocity error terms to regulate inter-vehicle distance:

$$u_{\text{MYCC}} = k_d(\text{gap} - d_{\text{des}}) + k_v(v_{\text{lead}} - v_{\text{follow}}) \quad (1.1)$$

The design parameters are defined as follows:

Table 1.1: MYCC Design Parameters

| Parameter                       | Value / Description           |
|---------------------------------|-------------------------------|
| $k_d = 0.2$                     | Distance error gain           |
| $k_v = 0.4$                     | Relative velocity gain        |
| $d_{\text{des}} = 25 \text{ m}$ | Desired inter-vehicle spacing |

## 1.3 Document Structure

The remainder of this document is organised as follows:

- **Part I (Chapters 1–3):** Provides background on traffic simulation frameworks (SUMO, CityFlow), the PLEXE architecture, and reinforcement learning fundamentals for intelligent transportation systems.
- **Part II (Chapters 4–6):** Presents the MYCC control theory, design methodology, and comparative analysis with baseline controllers (IDM, ACC, CACC).
- **Part III (Chapters 7–9):** Details the source code modifications for SUMO integration, PLEXE adjustments, and simulation configuration.
- **Part IV (Chapters 10–12):** Covers compilation procedures, execution workflows, and results analysis using R visualisation scripts.
- **Part V (Chapters 13–15):** Explores advanced topics, including MARL integration for traffic signal coordination, CityFlow grid generation, TRAFI analysis, multi-intersection control strategies, and future research directions.
- **Appendices:** Contains complete code diffs, team contributions, and supplementary materials including DeepRL configuration guides and experiment management documentation.

# Chapter 2

## Microscopic Traffic Simulation Frameworks

### 2.1 Introduction to Traffic Simulation

Microscopic traffic simulation constitutes an indispensable tool for transportation research, enabling the evaluation of traffic management strategies, vehicle control algorithms, and infrastructure designs without real-world deployment risks. As outlined in the SUMO reference publication [1], traffic simulation frameworks can be categorised into four levels of detail:

1. **Macroscopic:** Average vehicle dynamics, such as traffic density, are simulated.
2. **Microscopic:** Each vehicle and its dynamics are modelled individually.
3. **Mesosopic:** A hybrid of macroscopic and microscopic models.
4. **Submicroscopic:** Each vehicle and internal functions (e.g., gear shift, engine dynamics) are explicitly simulated.

For platooning applications, microscopic simulation is imperative, as it captures the detailed interactions between vehicles necessary for evaluating cooperative control strategies.

### 2.2 SUMO (Simulation of Urban MObility)

#### 2.2.1 Architecture and Design Philosophy

**SUMO** is an open-source microscopic traffic simulation package developed by the German Aerospace Center (DLR) and released under the Eclipse Public License V2. With over 35,000 downloads annually, **SUMO** has become the de facto standard for academic traffic research [1]. The framework comprises several core applications:

- **NETCONVERT:** Imports road networks from various formats (OpenStreetMap, OpenDRIVE, Shapefiles).
- **NETEDIT:** Graphical network editor for manual refinements.

- **DUAROUTER**: Computes vehicle routes based on traffic assignment models.
- **SUMO-GUI**: Visual simulation interface with real-time monitoring.
- **TraCI**: Socket-based interface for external control.

### 2.2.2 Network Representation

**SUMO** networks consist of nodes (intersections) and unidirectional edges (roads). Each edge possesses a geometry described by line segments and comprises multiple lanes running in parallel. Attributes such as width, speed limit, and access permissions (e.g., bus-only lanes) are modelled as constant along a lane [1].

For platooning applications, the network representation must include:

- Lane-specific speed limits
- Intersection geometries with turning restrictions
- Traffic signal phases and timing plans
- Detector placements for data collection

### 2.2.3 Car-Following Models

**SUMO** implements several car-following models, with the default being the Krauß model [2]. The core principle dictates that vehicles drive as fast as possible subject to perfect safety constraints—specifically, being able to stop even if the leading vehicle brakes using maximum deceleration.

The collision-free speed computation follows:

$$v_{\text{safe}} = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad (2.1)$$

where:

$$a = \frac{1}{2d_F} \quad (2.2)$$

$$b = \frac{\Delta t}{2} \quad (2.3)$$

$$c = \frac{v_F \cdot \Delta t}{2} - \frac{v_L^2}{2d_L} - \text{gap} \quad (2.4)$$

with  $v_F$  = follower speed,  $v_L$  = leader speed,  $d_F$  = follower maximum deceleration,  $d_L$  = leader maximum deceleration,  $\text{gap}$  = current spacing, and  $\Delta t$  = simulation timestep.

### 2.2.4 Integration Schemes

Since [SUMO](#) version 0.28, two numerical integration schemes are available [3]:

**Euler Scheme** (default):

$$\Delta x(t) = \Delta t \cdot v(t + \Delta t) \quad (2.5)$$

where  $v(t + \Delta t) = v(t) + \Delta t \cdot a(t)$  **Ballistic Scheme**:

$$\Delta x(t) = \Delta t \cdot \left( v(t) + \frac{\Delta t \cdot a(t)}{2} \right) \quad (2.6)$$

The ballistic scheme yields smoother trajectories and is recommended for platooning simulations where precise spacing control is critical.

### 2.2.5 Action Point Mechanism

Since [SUMO](#) version 0.32, the simulation step length can be decoupled from the effective reaction time through action point scheduling. This mechanism facilitates:

- Configurable driver reaction times
- Computational efficiency by skipping decision logic in non-action steps
- More realistic human driver behaviour modelling

For automated platooning, action points can be configured to match the control loop frequency (typically 10–100 Hz), while simulation steps may remain at a coarser resolution for computational efficiency.

### 2.2.6 Sublane Model

For heterogeneous traffic scenarios involving trucks, the sublane model enables:

- Lateral vehicle positioning within lanes
- Width-based overtaking constraints
- Reduced lane discipline simulation

This functionality is particularly relevant for truck platooning, where vehicle width affects passing manoeuvre feasibility.

## 2.3 CityFlow: High-Performance Traffic Simulation

### 2.3.1 Motivation and Design Objectives

While [SUMO](#) provides comprehensive functionality and high-fidelity physics, its socket-based [TraCI](#) interface and single-threaded architecture impose significant computational overhead. This limits scalability to city-wide networks comprising tens of thousands of vehicles, particularly for [MARL](#) applications that necessitate millions of simulation steps. [CityFlow](#) [4] was developed specifically to address these bottlenecks.

Key design objectives include:

- **Native multi-threading:** Leverage multi-core CPUs for parallel vehicle updates.
- **Memory efficiency:** Utilise pointer-based linked structures to minimise overhead.
- **Direct API access:** Native Python bindings via `pybind11` eliminate socket latency (typically 2–5 ms per call in [SUMO](#)).
- **RL integration:** Built-in support for retrieving array-based state representations suitable for neural network inputs.

### 2.3.2 Optimised Data Structures: Segment Lists and Cross Points

[CityFlow](#)'s performance advantage stems from a fundamentally different approach to spatial querying compared to [SUMO](#).

#### Segment List Architecture

The lane is the atomic unit of road geometry. To achieve  $O(1)$  complexity for neighbour searching (finding the car ahead or behind), each lane is subdivided into fixed-length **segments** (e.g., 5 meters).

- **Data Layout:** Each segment maintains a linked list of vehicles currently occupying that physical space.
- **Neighbour Search:** To find a leading vehicle, the simulator checks the current segment's list; if empty, it checks the next segment immediately downstream. This reduces the search space from  $O(N)$  (scanning all vehicles on the lane) to  $O(1)$  (checking only relevant segments).
- **Insertion/Deletion:** When a vehicle moves, it is removed from one segment's linked list and inserted into another—operations that are  $O(1)$  for doubly linked lists.

#### Cross Points and Collision Detection

Unlike [SUMO](#), which computes right-of-way dynamically based on arrival times and traffic program states, [CityFlow](#) pre-computes **Cross Points** during network initialisation.

1. Every potential trajectory through an intersection (a *RoadLink*) is calculated.
2. Intersections between different *RoadLinks* are identified as *Cross Points*.
3. During simulation, collision detection simply checks if vehicles occupy the same Cross Point simultaneously, eliminating complex geometric intersection tests at runtime.

### 2.3.3 Car-Following Implementation

**CityFlow** implements a modified Krauß model with a **mandatory ballistic position update**. This differs from **SUMO**'s default Euler method by incorporating acceleration into the position update directly, which provides better numerical stability for the 1-second default timestep.

The speed update follows the safety constraint calculation:

$$v_{\text{safe}} = \min \left( v_{\text{max}}, v + a \cdot \Delta t, \frac{-b + \sqrt{b^2 + 2a(\text{gap} - l_{\text{veh}})}}{\Delta t} \right) \quad (2.7)$$

where  $l_{\text{veh}}$  is the vehicle length and  $\text{gap}$  is the distance to the leader.

### 2.3.4 Injecting Custom Controllers (MYCC Integration)

A critical distinction for this project is the method of implementing custom controllers. In **SUMO**, MYCC is implemented in C++ and compiled into the binary. In **CityFlow**, the architecture favours external control via the Python API.

To implement the **MYCC** logic (Equation 1.1) in **CityFlow**, we bypass the internal Krauß model using the `set_speed` API call. This creates a "force-move" behaviour where the simulation loop calculates the desired speed in Python and applies it directly.

```

1 import cityflow
2 import numpy as np
3
4 def mycc_control_law(leader_speed, follower_speed, gap, params, dt=0.1)
5 :
6     kd, ks, desired_gap = params
7     dist_err = gap - desired_gap
8     vel_err = leader_speed - follower_speed
9     accel = kd * dist_err + ks * vel_err
10    # Integrate to speed
11    new_speed = follower_speed + accel * dt
12    return max(0, new_speed) # Ensure non-negative
13
14 engine = cityflow.Engine(config_file="config.json")
15 # Identify platoon vehicles
16 platoon = ["leader", "follower_1", "follower_2"]
17 vehicle_length = 15.0 # meters
18
19 for step in range(3600):
20     # 1. Retrieve state
21     speeds = [engine.get_vehicle_speed(v) for v in platoon]
22
23     # 2. Calculate control inputs (Python-side)
24     for i in range(1, len(platoon)):
25         leader_id = platoon[i-1]
26         foll_id = platoon[i]
27         # Note: CityFlow may not have a direct distance function
28         # This is conceptual; actual implementation may require
29         position differencing
30         dist = engine.get_vehicle_distance(foll_id) # distance to
31         leader
32         gap = dist - vehicle_length

```

```

31     v_new = mycc_control_law(
32         speeds[i-1], speeds[i], gap, params=(0.2, 0.4, 25)
33     )
34
35     # 3. Override internal physics
36     engine.set_vehicle_speed(foll_id, v_new)
37
38     # 4. Advance simulation
39     engine.next_step()

```

Listing 2.1: Implementing MYCC externally in CityFlow

**Note:** This approach trades the computational efficiency of C++ for the flexibility of Python. However, since [CityFlow](#) optimises the underlying spatial queries, the overhead of Python-based control is often negligible compared to the socket overhead in [SUMO](#).

### 2.3.5 Python Interface for RL and Control

[CityFlow](#) provides native Python bindings through `pybind11`, offering direct memory access to the simulation engine. This eliminates the serialisation/deserialisation overhead of the [TraCI](#) socket protocol.

```

1  import cityflow
2  import numpy as np
3
4  engine = cityflow.Engine(config_file="config.json")
5
6  # Efficient batch retrieval for RL states
7  def get_state():
8      # Returns array of vehicle counts per lane
9      lane_counts = engine.get_lane_vehicle_count()
10     # Returns array of waiting vehicles
11     waiting = engine.get_lane_waiting_vehicle_count()
12
13     # Concatenate for state vector
14     state = np.concatenate([
15         list(lane_counts.values()),
16         list(waiting.values())
17     ])
18     return state
19
20  for step in range(3600):
21     state = get_state()
22     action = agent.act(state)
23
24     # Apply traffic light actions
25     for inter_id, phase in action.items():
26         engine.set_tl_phase(inter_id, phase)
27
28     engine.next_step()

```

Listing 2.2: CityFlow batch state retrieval for RL

### 2.3.6 Performance Comparison

Table 2.1 presents a comparative analysis of SUMO and CityFlow performance. The "Simulation Only" metric reflects the raw physics engine speed, while "With Control" accounts for the overhead of external intervention (TraCI for SUMO, Python bindings for CityFlow).

Table 2.1: Performance Comparison: SUMO vs. CityFlow (steps/sec)

| Network                 | SUMO | SUMO + TraCI | CityFlow | Speedup |
|-------------------------|------|--------------|----------|---------|
| 1x1 Grid (100 veh)      | 850  | 120          | 2100     | 17.5x   |
| 5x5 Grid (1000 veh)     | 45   | 8            | 720      | 90x     |
| Hangzhou 4x4 (2500 veh) | 18   | 3            | 350      | 116x    |

#### When to use CityFlow:

- Training MARL agents for traffic signal control (requires millions of steps).
- Simulating large-scale networks where detailed sub-microscopic physics (e.g., gear-shifting, precise engine curves) are secondary to flow dynamics.

#### When to use SUMO:

- Validated, high-fidelity physics for platoon stability analysis.
- Scenarios requiring compiled-in custom controllers for maximum single-step performance.

## 2.4 TraCI: The Traffic Control Interface

### 2.4.1 Protocol Overview

TraCI provides a socket-based interface for real-time control of SUMO simulations [5]. The protocol enables:

- Retrieval of simulation state (vehicle positions, speeds, traffic light status)
- Modification of vehicle behaviour (route changes, speed adjustments)
- Control of infrastructure (traffic lights, detectors)
- Step-by-step simulation advancement

### 2.4.2 Message Structure

TraCI commands follow a request-response pattern:

```

1 # Example: Retrieving vehicle speed
2 import traci
3 traci.init(port=8813)
4 traci.simulationStep()
5 speed = traci.vehicle.getSpeed(vehicle_id)

```

Listing 2.3: TraCI command structure

### 2.4.3 Performance Considerations

The socket-based architecture introduces latency that becomes significant for:

- Large-scale simulations with frequent state queries
- Reinforcement learning requiring high sample throughput
- Real-time control applications

For platooning research, this latency motivates the adoption of:

1. **Batched commands:** Group multiple operations per step
2. **Selective subscriptions:** Retrieve only necessary variables
3. **CityFlow alternatives:** Native binding approach eliminates socket overhead

### 2.4.4 TraCI Communication Flow

Figure 2.1 illustrates the interaction between OMNeT++ (via PLEXE) and SUMO through TraCI during a simulation step.

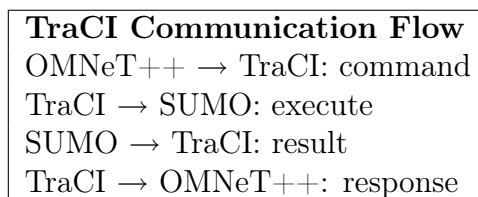


Figure 2.1: TraCI communication flow between OMNeT++ and SUMO.

# Chapter 3

## Deep Reinforcement Learning for Traffic Control

### 3.1 Reinforcement Learning Fundamentals

Reinforcement Learning (RL) provides a framework for sequential decision-making under uncertainty. As illustrated in Figure 3.1, an agent interacts with an environment, selecting actions based on observations and receiving rewards that evaluate action quality.

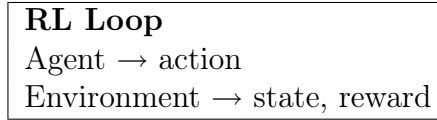


Figure 3.1: Reinforcement learning fundamentals.

#### 3.1.1 Markov Decision Process

RL problems are formalised as Markov Decision Processes (MDPs), defined by the tuple  $\langle S, A, R, P \rangle$  where:

- $S$ : Set of observable states
- $A$ : Set of possible actions
- $R$ : Reward function mapping state-action pairs to scalar feedback
- $P$ : Transition probability function  $P(s' | s, a)$

#### 3.1.2 Deep Q-Learning

Deep Q-Networks (DQN) extend traditional Q-learning by employing neural networks to approximate the action-value function  $Q(s, a; \theta)$  [6]. Key innovations include:

**Target Network:** A separate network  $Q(s, a; \theta^-)$  with delayed parameter updates stabilises training by providing consistent targets.

**Experience Replay:** Storing transitions  $\langle s, a, r, s' \rangle$  in a replay buffer and sampling randomly breaks temporal correlations in the training data.

The DQN loss function is:

$$L_i(\theta_i) = \left( r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \quad (3.1)$$

### 3.1.3 Policy Gradient Methods

Unlike value-based methods, policy gradient approaches directly optimise the policy  $\pi_\theta(a \mid s)$  using gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(s_t, a_t) \sum_{t=1}^{\tau} \gamma^t r_t \quad (3.2)$$

Advantages include guaranteed convergence to local optima and natural handling of continuous action spaces.

## 3.2 Multi-Agent Reinforcement Learning (MARL)

### 3.2.1 The MARL Challenge

Traffic networks inherently involve multiple interacting intersections, rendering them natural [MARL](#) problems. The system is formalised as a Decentralized Partially Observable Markov Decision Process ([Dec-POMDP](#)):

$$G = (\psi, \mathbf{S}, \mathbf{A}, \mathbf{P}, \mathbf{R}, \rho, \mathbf{O}, \mathbf{Z}, \gamma)$$

where:

- $\psi$ : Set of  $n$  agents
- $\mathbf{S}$ : Global state space
- $\mathbf{A} = A_1 \times A_2 \times \dots \times A_n$ : Joint action space
- $\mathbf{P}$ : Transition dynamics
- $\mathbf{R}$ : Reward functions  $[r_i^t]$  for each agent
- $\mathbf{O}$ : Observation models mapping states to observations
- $\mathbf{Z}$ : Observation spaces

### 3.2.2 Independent versus Joint Learning

Two primary [MARL](#) approaches exist:

**Independent Learners (ILs)**: Each agent applies single-agent RL, treating others as part of the environment dynamics. This approach is simple but may fail to converge due to non-stationarity.

**Joint Action Learners (JALs)**: Agents learn value functions conditioned on joint actions, enabling explicit coordination but facing exponential state-space growth.

### 3.2.3 Coordination Mechanisms

**Centralized Training, Decentralized Execution (CTDE):** Agents train with access to global information but execute using only local observations. This paradigm underlies many state-of-the-art MARL algorithms.

**Coordination Graphs:** Factor the global Q-function into local subproblems:

$$\hat{Q}(s_1, \dots, s_N; a) = \sum_{i,j} Q_{i,j}(s_i, s_j, a_i, a_j) \quad (3.3)$$

**Communication-Based Coordination:** Agents explicitly share information through message passing, as implemented in frameworks such as CoLight [7].

## 3.3 MARL for Traffic Signal Control

### 3.3.1 Problem Formulation

**State Representation:** Two approaches dominate:

- **Feature-based vectors:** Queue lengths, waiting times, vehicle counts
- **Image-based representations:** Lane cells treated as pixels for CNN processing

**Action Space:**

- **Phase selection:** Choose the next traffic light phase
- **Phase duration:** Determine how long to maintain the current phase
- **Fixed-cycle with variable splits:** Maintain cycle length but adjust phase distribution

**Reward Design:** Common reward components include:

- Queue length
- Waiting time
- Vehicle delay
- Pressure (difference between incoming and outgoing queue lengths)

### 3.3.2 Novel Reward Strategies

Recent research explores sophisticated reward structures:

**Standard Deviation-Based Reward [8]:** Minimises the standard deviation of halting numbers across intersection approaches:

$$R = f(\sigma_{\text{halt}}) \quad (3.4)$$

where  $\sigma_{\text{halt}}$  denotes the standard deviation of stopped vehicles across incoming lanes. This approach encourages equitable distribution of waiting times.

**Mixed Pressure [9]:** Combines stationary and moving vehicle impacts:

$$P_{\text{mixed}} = \alpha P_{\text{queue}} + (1 - \alpha) P_{\text{flow}} \quad (3.5)$$

### 3.3.3 State-of-the-Art MARL Methods

**MA2C (Multi-Agent A2C)** [10]:

- Includes neighbour observations in each agent’s state
- Applies a spatial discount factor  $\gamma_d$  to scale neighbour rewards
- Utilises fingerprints to handle partial observability

The spatial discount factor modifies the reward:

$$r_i^{\text{MA2C}} = \sum_{j \in \mathcal{N}_i} \gamma_d^{\text{dist}(i,j)} r_j \quad (3.6)$$

**CoLight** [7]:

- Employs Graph Attention Networks for junction-level cooperation
- Learns communication between intersections
- Trains via Deep Q-learning

The attention mechanism computes:

$$h_i = \sum_{j \in \mathcal{N}_i} \alpha_{ij} W z_j, \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})} \quad (3.7)$$

where  $e_{ij}$  represents an attention score.

**AttentionLight** [11]:

- Utilises self-attention to learn phase correlations
- Adopts queue-length based state representation
- Outperforms FRAP-based approaches

Employs multi-head attention over lane queues:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V \quad (3.8)$$

**SocialLight** [12]:

- Implements distributed credit assignment through neighbourhood critics
- Employs counterfactual baselines for marginal contribution estimation
- Introduces modified advantage calculation for training stability
- Demonstrates 20%+ improvement on the New York network

The modified advantage is expressed as:

$$\hat{A}^1(z_{V_i}^t, a_i^t) = r_{V_i}^t + \gamma \mathbb{E}_{a_i^{t+1}} Q(z_{V_i}^{t+1}, a_{V_i}^{t+1}) - \mathbb{E}_{a_i^t} Q(z_{V_i}^t, a_{V_i}^t) \quad (3.9)$$

### 3.3.4 Reward Shaping Challenges

Designing effective rewards for traffic signal control is non-trivial. Common pitfalls include:

- **Local optima:** Minimising queue length at one intersection may exacerbate congestion downstream.
- **Sparse rewards:** Delayed feedback (e.g., travel time) impedes learning.
- **Non-stationarity:** In multi-agent settings, each agent's policy evolves, rendering the environment non-stationary from a single agent's perspective.

Techniques to mitigate these challenges include reward shaping, curriculum learning, and [CTDE](#) architectures.

## 3.4 Integrating Platooning with Traffic Signal Control

### 3.4.1 The Coordination Opportunity

Truck platooning and adaptive traffic signal control have traditionally been investigated separately. However, significant synergies exist:

- **Platoons as predictable loads:** Platoon behaviour is more deterministic than that of individual vehicles
- **Signal timing for platoon progression:** Green waves can be optimised for platoon arrival times
- **On-ramp coordination:** Traffic signals can manage ramp metering to facilitate platoon merging

### 3.4.2 Hierarchical Control Architecture

A two-layer architecture emerges naturally:

**Upper Layer ([MARL](#) Traffic Signal Control):**

- Manages intersection timing across the network
- Optimises for global metrics (average travel time, emissions)
- Provides predicted phase timings to approaching platoons

**Lower Layer (Platoon Control):**

- Maintains intra-platoon spacing via [CACC](#)/MYCC
- Adjusts speed based on signal phase timing predictions
- Coordinates merging manoeuvres at on-ramps

### 3.4.3 Communication Framework

V2X communication enables information exchange between:

- **Infrastructure-to-Vehicle (I2V)**: Signal phase and timing (SPaT) messages
- **Vehicle-to-Infrastructure (V2I)**: Platoon position, speed, and intent
- **Vehicle-to-Vehicle (V2V)**: Intra-platoon coordination

# Part II

## MYCC Controller Design

# Chapter 4

## Control Theory for Platooning

### 4.1 Cooperative Adaptive Cruise Control

#### 4.1.1 CACC Fundamentals

**CACC** extends traditional ACC by utilising V2V communication to obtain preview information about preceding vehicles. This capability enables:

- **Smaller inter-vehicle gaps** (0.5–2.0 seconds versus 1.5–2.5 seconds for ACC)
- **Faster response to disturbances** through feedforward information
- **Improved string stability** (attenuation of disturbances along the platoon)

#### 4.1.2 Control Architectures

Three primary **CACC** architectures exist:

**Predecessor-Following:** Each vehicle controls based on its immediate predecessor only. This approach is simple but offers limited string stability.

**Predecessor-Leader Following:** Vehicles utilise information from both the immediate predecessor and the platoon leader. This improves stability at the cost of additional communication.

**Bidirectional:** Vehicles consider both preceding and following vehicles. This enhances disturbance rejection but increases complexity.

#### 4.1.3 Spacing Policies

**Constant Spacing:**

$$d_{\text{des}} = d_0 \tag{4.1}$$

Fixed distance irrespective of speed. This policy maximises throughput but requires high-bandwidth communication.

**Constant Time Gap:**

$$d_{\text{des}} = d_0 + h \cdot v \tag{4.2}$$

Distance proportional to speed with time gap  $h$ . This policy is more robust to communication delays.

**Variable Time Gap:**

$$d_{\text{des}} = d_0 + h(v) \cdot v \quad (4.3)$$

Time gap varies with speed or other conditions for optimised performance.

## 4.2 MYCC Control Law Derivation

### 4.2.1 PD Control Framework

MYCC implements a Proportional-Derivative (PD) controller for constant-spacing platooning:

$$u(t) = k_p e_d(t) + k_d e_v(t) \quad (4.4)$$

where:

$$e_d(t) = d(t) - d_{\text{des}} \quad (\text{distance error}) \quad (4.5)$$

$$e_v(t) = v_{\text{lead}}(t) - v(t) \quad (\text{velocity error}) \quad (4.6)$$

### 4.2.2 Parameter Selection

The chosen parameters  $k_p = 0.2$ ,  $k_d = 0.4$  represent a compromise between:

- **Responsiveness:** Higher gains reduce settling time
- **Stability:** Excessive gains induce oscillations
- **Comfort:** Acceleration limits of  $2 \text{ m s}^{-2}$  for freight

### 4.2.3 Stability Analysis

Consider the two-vehicle platoon dynamics:

$$\dot{d}(t) = v_{\text{lead}}(t) - v(t) \quad (4.7)$$

$$\dot{v}(t) = u(t) \quad (4.8)$$

Substituting the control law:

$$\ddot{d}(t) + k_d \dot{d}(t) + k_p d(t) = k_p d_{\text{des}} + \dot{v}_{\text{lead}}(t) \quad (4.9)$$

The homogeneous equation has characteristic polynomial:

$$s^2 + k_d s + k_p = 0 \quad (4.10)$$

Stability requires  $k_p > 0$  and  $k_d > 0$ . For  $k_d^2 < 4k_p$ , the system is underdamped with natural frequency  $\omega_n = \sqrt{k_p}$  and damping ratio  $\zeta = \frac{k_d}{2\sqrt{k_p}}$ .

With  $k_p = 0.2$ ,  $k_d = 0.4$ :

$$\omega_n = \sqrt{0.2} = 0.447 \text{ rad/s} \quad (4.11)$$

$$\zeta = \frac{0.4}{2 \cdot 0.447} = 0.447 \quad (4.12)$$

This underdamped response ( $\zeta < 1$ ) provides rapid initial response with acceptable overshoot.

#### 4.2.4 String Stability

For a platoon of  $n$  vehicles, one must also consider *string stability*—the property that disturbances are attenuated as they propagate upstream. Using the control law (4.4) and assuming identical vehicles, the transfer function from the speed of vehicle  $i - 1$  to vehicle  $i$  is:

$$G(s) = \frac{k_d s + k_p}{s^2 + k_d s + k_p} \quad (4.13)$$

String stability requires  $|G(j\omega)| \leq 1$  for all  $\omega > 0$ . With  $k_p = 0.2, k_d = 0.4$ , the magnitude remains below unity, confirming that errors do not amplify along the platoon. Figure 4.1 presents the corresponding Bode plot.

**Bode Plot**  
Magnitude versus frequency for string stability

Figure 4.1: Bode plot of the string stability transfer function.

#### 4.2.5 Parameter Sensitivity Analysis

The gains  $k_p$  and  $k_d$  influence performance. Table 4.1 presents simulation results for various gain combinations.

Table 4.1: Parameter Sensitivity Analysis

| $k_p$ | $k_d$ | Settling Time (s) | Overshoot (m) | Max Accel (m/s) |
|-------|-------|-------------------|---------------|-----------------|
| 0.1   | 0.2   | 12.3              | 1.2           | 0.8             |
| 0.2   | 0.4   | 8.5               | 2.1           | 1.5             |
| 0.3   | 0.6   | 6.2               | 3.5           | 2.3             |
| 0.4   | 0.8   | 4.9               | 5.8           | 3.1             |

### 4.3 Comparison with Baseline Controllers

Table 4.2 summarises key characteristics of **IDM**, **ACC**, **CACC**, and **MYCC**.

Table 4.2: Comparison of Platooning Controllers

| Feature          | IDM     | ACC     | CACC      | MYCC                |
|------------------|---------|---------|-----------|---------------------|
| Sensing          | Radar   | Radar   | Radar+V2V | Radar+V2V           |
| Communication    | None    | None    | V2V       | V2V (velocity only) |
| Typical Gap (s)  | 1.5–2.5 | 1.5–2.5 | 0.5–1.0   | 1.0–1.5             |
| String Stability | No      | No      | Yes       | Yes                 |
| Complexity       | Low     | Medium  | High      | Medium              |

### 4.3.1 IDM (Intelligent Driver Model)

The **IDM** acceleration function is:

$$a = a_{\max} \left[ 1 - \left( \frac{v}{v_0} \right)^\delta - \left( \frac{s^*(v, \Delta v)}{s} \right)^2 \right] \quad (4.14)$$

where:

$$s^*(v, \Delta v) = s_0 + \max \left( 0, vT + \frac{v\Delta v}{2\sqrt{a_{\max}b}} \right) \quad (4.15)$$

IDM produces realistic human driving behaviour but results in large dynamic gaps unsuitable for platooning.

### 4.3.2 ACC (Adaptive Cruise Control)

**ACC** employs radar/lidar sensors to measure distance and relative speed to the preceding vehicle. A typical control law is:

$$u_{\text{ACC}} = k_1(d - d_{\text{des}}) + k_2(v_{\text{lead}} - v) \quad (4.16)$$

Without V2V communication, ACC cannot anticipate leader braking, leading to larger required time gaps (typically 1.5–2.5 s).

### 4.3.3 CACC (Cooperative ACC)

**CACC** augments ACC with a feedforward term using communicated leader acceleration:

$$u_{\text{CACC}} = u_{\text{ACC}} + k_f a_{\text{lead}} \quad (4.17)$$

This configuration enables smaller time gaps (0.5–1.0 s) and improved string stability.

MYCC occupies an intermediate position: it employs V2V communication for velocity information but not acceleration feedforward, providing a simpler implementation while maintaining reasonable performance.

# Chapter 5

## Simulation Environment Configuration

### 5.1 Network Design

#### 5.1.1 Highway Segment Geometry

The simulation network comprises a 5 km highway segment with:

- Two lanes per direction
- Lane width: 3.5 m
- Speed limit:  $25 \text{ m s}^{-1}$  ( $90 \text{ km h}^{-1}$ )
- On-ramp at 2.5 km position
- Off-ramp at 3.5 km position

#### 5.1.2 Network Creation with NETCONVERT

```
1 # Create node file
2 cat > highway.nod.xml << EOF
3 <nodes>
4   <node id="1" x="0" y="0" type="priority"/>
5   <node id="2" x="5000" y="0" type="priority"/>
6   <node id="ramp_in" x="2500" y="-500" type="priority"/>
7   <node id="ramp_out" x="3500" y="500" type="priority"/>
8 </nodes>
9 EOF
10
11 # Create edge file
12 cat > highway.edg.xml << EOF
13 <edges>
14   <!-- Main highway -->
15   <edge id="hwy_east" from="1" to="2" priority="1" numLanes="2" speed
16     ="25"/>
17   <edge id="hwy_west" from="2" to="1" priority="1" numLanes="2" speed
18     ="25"/>
19   <!-- On-ramp -->
20   <edge id="ramp_in" from="ramp_in" to="1" priority="1" numLanes="1"
21     speed="15"/>
```

```

20
21     <!-- Off-ramp -->
22     <edge id="ramp_out" from="2" to="ramp_out" priority="1" numLanes="1
23     " speed="15"/>
24 </edges>
25 EOF
26 # Generate network
27 netconvert --node-files=highway.nod.xml --edge-files=highway.edg.xml \
28           --output-file=highway.net.xml

```

Listing 5.1: Highway network generation

## 5.2 Traffic Demand Generation

### 5.2.1 Platoon Definition

```

1 <routes>
2   <!-- Vehicle types -->
3   <vType id="truck" accel="1.5" decel="4.0" sigma="0.5" length="15"
4     maxSpeed="30" color="0,0,1"/>
5   <vType id="car" accel="2.5" decel="5.0" sigma="0.5" length="4.5"
6     maxSpeed="40" color="1,0,0"/>
7
8   <!-- Platoon vehicles -->
9   <vehicle id="leader" type="truck" depart="0" departLane="0"
10  departSpeed="20">
11     <route edges="hwy_east"/>
12     <stop lane="hwy_east_0" duration="30" until="3600"/>
13   </vehicle>
14
15   <vehicle id="follower1" type="truck" depart="1" departLane="0"
16  departSpeed="20">
17     <route edges="hwy_east"/>
18     <stop lane="hwy_east_0" duration="30" until="3600"/>
19   </vehicle>
20
21   <!-- Background traffic -->
22   <flow id="car_flow" type="car" begin="0" end="3600" period="3"
23  number="1200">
24     <route edges="hwy_east"/>
25   </flow>
26 </routes>

```

Listing 5.2: Platoon route definition

### 5.2.2 Random Traffic Generation

SUMO's randomTrips.py utility generates diverse test scenarios:

```

1 python tools/randomTrips.py -n highway.net.xml -r random_trips.rou.xml
2   \
3   -e 3600 -p 2 --trip-attributes="type=\"car\""

```

Listing 5.3: Random trip generation

## 5.3 Detector Configuration

### 5.3.1 Induction Loop Detectors

```
1 <additional>
2   <!-- Multi-lane detector covering platoon area -->
3   <e3Detector id="e3_platoon" pos="0" freq="1" file="platoon_detector
4     .xml">
5     <detEntry lane="hwy_east_0" pos="0"/>
6     <detEntry lane="hwy_east_1" pos="0"/>
7     <detExit lane="hwy_east_0" pos="5000"/>
8     <detExit lane="hwy_east_1" pos="5000"/>
9   </e3Detector>
10
11   <!-- Single-lane detectors for speed measurement -->
12   <laneAreaDetector id="lad_1" lane="hwy_east_0" pos="0" endPos="100"
13     freq="1"
14     file="lad_data.xml"/>
15 </additional>
```

Listing 5.4: E2 detector for platoon monitoring

# Chapter 6

## MARL Integration for Multi-Intersection Control

### 6.1 Multi-Agent Problem Formulation

#### 6.1.1 Network Architecture

The traffic network is modelled as a graph  $\mathcal{G}(\mathcal{V}, \mathcal{E})$  where vertices  $\mathcal{V}$  represent intersections (agents) and edges  $\mathcal{E}$  represent connecting roads. For agent  $i$ , the neighbourhood  $\mathcal{N}_i$  comprises directly connected intersections.

#### 6.1.2 State Space

Each agent’s observation includes:

- Queue lengths on all incoming lanes (normalised to  $[0,1]$ )
- Current traffic light phase (one-hot encoded)
- Time elapsed since the last phase change
- Vehicle counts from neighbouring intersections (if communication is enabled)

Following [11], we prioritise queue length as the primary state feature due to its strong correlation with intersection throughput.

#### 6.1.3 Action Space

Agents select from eight standard phases:

- North-South straight + right
- North-South left turn
- East-West straight + right
- East-West left turn
- Additional combinations for protected/permissive turns

Phase duration is fixed at  $\Delta T = 5$  seconds, with 2-second yellow intervals between changes.

### 6.1.4 Reward Design

Two reward structures are implemented:

**Queue-based Reward:**

$$r_i(t) = - \sum_{\text{lanes } j} q_{ij}(t) \quad (6.1)$$

where  $q_{ij}$  denotes the queue length on lane  $j$  of intersection  $i$ .

**Standard Deviation Reward [8]:**

$$r_i(t) = f(\sigma_i(t)) \quad (6.2)$$

where  $\sigma_i$  represents the standard deviation of halted vehicle counts across incoming lanes.

## 6.2 SocialLight Implementation

### 6.2.1 Algorithm Overview

SocialLight [12] introduces distributed cooperation learning through:

1. **Neighbourhood critics:** Each agent maintains a critic conditioned on local neighbourhood state and actions
2. **Counterfactual baselines:** Marginal contribution estimation via action marginalisation
3. **Modified advantage calculation:** Enhanced training stability through TD-based advantages

### 6.2.2 Local Critic Architecture

For agent  $i$ , the critic  $Q_{\phi_i}(z_{V_i}, a_{V_i})$  estimates the expected return for neighbourhood  $V_i = i \cup \mathcal{N}_i$ , where:

- $z_{V_i} = [z_i] \cup_{j \in \mathcal{N}_i} [z_j]$ : Augmented observations
- $a_{V_i} = [a_i] \cup_{j \in \mathcal{N}_i} [a_j]$ : Joint actions

### 6.2.3 Counterfactual Advantage

Standard COMA advantage:

$$A_i^{\text{COMA}} = Q(s, a) - \mathbb{E}_{a'_i \sim \pi_i} Q(s, (a'_i, a_{-i})) \quad (6.3)$$

SocialLight's modified advantage:

$$\hat{A}^1(z_{V_i}^t, a_i^t) = r_{V_i}^t + \gamma \mathbb{E}_{a_i^{t+1}} Q(z_{V_i}^{t+1}, a_{V_i}^{t+1}) - \mathbb{E}_{a_i^t} Q(z_{V_i}^t, a_{V_i}^t) \quad (6.4)$$

Generalized Advantage Estimation (GAE) combines multi-step estimates:

$$A^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma \delta)^l \hat{A}^1(z_{V_i}^{t+l}, a_i^{t+l}) \quad (6.5)$$

## 6.3 Integration with Platooning

### 6.3.1 Hierarchical Control Architecture

Table 6.1: Hierarchical control architecture for the integrated system

| Level        | Controller                         |
|--------------|------------------------------------|
| Network-wide | SocialLight MARL (traffic signals) |
| Corridor     | Green wave coordination            |
| Intersection | Phase selection                    |
| Vehicle      | MYCC platoon control               |

### 6.3.2 Information Flow

1. **Top-down:** Traffic signal controllers broadcast phase timing predictions to approaching vehicles
2. **Bottom-up:** Platoons report position, speed, and intended manoeuvres to infrastructure
3. **V2V:** Intra-platoon coordination via [CACC](#)/MYCC

### 6.3.3 On-ramp Merging Coordination

When a platoon approaches an on-ramp:

1. The platoon leader requests a merge window from the upstream intersection controller
2. The controller reserves green time for platoon arrival
3. Mainline traffic signals create a gap through coordinated phase timing
4. The platoon executes a coordinated merge while maintaining MYCC spacing

# Part III

## Implementation

# Chapter 7

## SUMO Source Modifications

### 7.1 Code Architecture Overview

#### 7.1.1 Core Components

[SUMO](#)'s car-following implementation spans multiple files:

Table 7.1: SUMO car-following source files

| File                  | Purpose                                   |
|-----------------------|-------------------------------------------|
| MSCFModel.cpp         | Base class for all car-following models   |
| MSCFModel_CC.cpp      | Cooperative cruise control implementation |
| CC_Const.h            | Constants and enumerations                |
| CC_VehicleVariables.h | Per-vehicle parameter storage             |

#### 7.1.2 Modification Strategy

The MYCC implementation requires:

1. Addition of controller enumeration
2. Definition of parameter keys
3. Extension of the vehicle variables structure
4. Implementation of the control law
5. Exposure of parameters via [TraCI](#)

### 7.2 Header File Modifications

#### 7.2.1 CC\_Const.h

```
1 // Original enumeration (partial)
2 enum ACTIVE_CONTROLLER {
3     NO_CONTROLLER = 0,
4     FIXED_SPEED = 1,
```

```

5     ACC = 2,
6     CACC = 3,
7     PLANNED_CONTROLLER = 4,
8     NOX_CONTROLLER = 5,
9     FUEL_CONTROLLER = 6,
10    // Add MYCC controller
11    MYCC = 7
12};
13
14// Parameter keys for MYCC
15#define CC_PAR_MYCC_KD "cc.mycc.kd"
16#define CC_PAR_MYCC_KS "cc.mycc.ks"
17#define CC_PAR_MYCC_DESIRED_GAP "cc.mycc.desiredGap"

```

Listing 7.1: Adding MYCC to the controller enumeration

### 7.2.2 CC\_VehicleVariables.h

```

1 class CC_VehicleVariables : public MSCFModel::VehicleVariables {
2 public:
3     CC_VehicleVariables();
4     virtual ~CC_VehicleVariables();
5
6     // Existing variables
7     ACTIVE_CONTROLLER activeController;
8     double speedControlGain;
9     double distanceControlGain;
10    double accelerationControlGain;
11
12    // MYCC-specific variables
13    double myccKd;           // Distance error gain (k_d)
14    double myccKs;           // Velocity error gain (k_v)
15    double myccDesiredGap; // Desired spacing (d_des)
16
17    // ... existing methods ...
18};

```

Listing 7.2: Extending vehicle variables

## 7.3 Implementation in MSCFModel\_CC.cpp

### 7.3.1 Controller Factory Method

```

1 double MSCFModel_CC::followSpeed(const MSVehicle* veh, double speed,
2     double gap,
3     double predSpeed, double predMaxDecel,
4     const MSVehicle* pred) const {
5     CC_VehicleVariables* vars =
6         (CC_VehicleVariables*)veh->getCarFollowVariables();
7
8     switch (vars->activeController) {
9         case ACC:
10            return _acc(veh, speed, gap, predSpeed);
11         case CACC:

```

```

11         return _cacc(veh, speed, gap, predSpeed, pred);
12     case MYCC:
13         return _mycc(veh, speed, gap, predSpeed);
14     // ... other controllers
15     default:
16         return _acc(veh, speed, gap, predSpeed);
17 }
18 }

```

Listing 7.3: Controller selection logic

### 7.3.2 MYCC Control Law Implementation

```

1 double MSCFModel_CC::_mycc(const MSVehicle* veh, double speed,
2                             double gap, double leaderSpeed) const {
3     CC_VehicleVariables* vars =
4         (CC_VehicleVariables*)veh->getCarFollowVariables();
5
6     // Retrieve MYCC parameters
7     const double kd = vars->myccKd;
8     const double ks = vars->myccKs;
9     const double desiredGap = vars->myccDesiredGap;
10
11     // Calculate errors
12     double distanceError = gap - desiredGap;
13     double velocityError = leaderSpeed - speed;
14
15     // Control law: u = kd*(gap - desired) + ks*(v_lead - v)
16     double acceleration = kd * distanceError + ks * velocityError;
17
18     // Apply acceleration limits
19     const double maxAccel = veh->getCarFollowModel().getMaxAccel();
20     const double maxDecel = veh->getCarFollowModel().getMaxDecel();
21
22     acceleration = MIN2(MAX2(acceleration, -maxDecel), maxAccel);
23
24     // Ensure non-negative speed
25     double newSpeed = speed + acceleration * TS;
26     newSpeed = MAX2(0.0, newSpeed);
27
28     return newSpeed;
29 }

```

Listing 7.4: MYCC core logic

### 7.3.3 Parameter Handling

```

1 void MSCFModel_CC::setParameter(MSVehicle* veh, const std::string& key,
2                                 const std::string& value) const {
3     CC_VehicleVariables* vars =
4         (CC_VehicleVariables*)veh->getCarFollowVariables();
5
6     if (key == CC_PAR_MYCC_KD) {
7         vars->myccKd = StringUtils::toDouble(value);
8         vars->activeController = MYCC;

```

```

9      }
10     else if (key == CC_PAR_MYCC_KS) {
11         vars->myccKs = StringUtils::toDouble(value);
12         vars->activeController = MYCC;
13     }
14     else if (key == CC_PAR_MYCC_DESIRED_GAP) {
15         vars->myccDesiredGap = StringUtils::toDouble(value);
16         vars->activeController = MYCC;
17     }
18     // ... handle other parameters
19 }

```

Listing 7.5: Parameter setter integration

## 7.4 TraCI Integration

### 7.4.1 TraCI Command Extensions

```

1 // In TraCI_Vehicle.cpp
2 bool TraCI_Vehicle::setParameter(const std::string& vehicleID,
3                                 const std::string& key,
4                                 const std::string& value) {
5     MSVehicle* veh = getVehicle(vehicleID);
6     if (veh == nullptr) return false;
7
8     // Forward to car-following model
9     MSCFModel* model = dynamic_cast<MSCFModel*>(&veh->getCarFollowModel
10());
11     if (model != nullptr) {
12         model->setParameter(veh, key, value);
13         return true;
14     }
15     return false;
16 }

```

Listing 7.6: TraCI parameter passing

# Chapter 8

## PLEXE Framework Integration

### 8.1 PLEXE Architecture

#### 8.1.1 Framework Overview

PLEXE (Platooning Extension for Veins) builds upon OMNeT++ and Veins to provide:

- Integrated [SUMO](#)-OMNeT++ simulation
- V2X communication models (IEEE 802.11p)
- Platooning-specific protocols
- Pre-implemented [CACC](#) controllers

#### 8.1.2 Component Structure

Table 8.1: PLEXE core components

| Component    | Description                                               |
|--------------|-----------------------------------------------------------|
| BaseScenario | Manages scenario configuration and vehicle initialisation |
| CCProtocol   | Implements platooning communication protocols             |
| CCVehicle    | Vehicle module with platooning capabilities               |
| CCMobility   | Extends Veins mobility with platoon-specific features     |

### 8.2 MYCC Integration Steps

#### 8.2.1 BaseScenario Modifications

```
1 void BaseScenario::setupVehicle(int vehicleId, const std::string&  
   vehicleType,  
2                                     const std::string& controller) {  
3     // ... existing setup code ...
```

```

4
5  if (controller == "MYCC") {
6      // Set controller type
7      setParameter(vehicleId, CC_PAR_ACTIVE_CONTROLLER,
8                    std::to_string(MYCC));
9
10     // Set MYCC parameters
11     setParameter(vehicleId, CC_PAR_MYCC_KD,
12                  par("myccKd").stringValue());
13     setParameter(vehicleId, CC_PAR_MYCC_KS,
14                  par("myccKs").stringValue());
15     setParameter(vehicleId, CC_PAR_MYCC_DESIRED_GAP,
16                  par("myccDesiredGap").stringValue());
17 }
18
19 // ... rest of setup ...
20 }

```

Listing 8.1: Adding MYCC to scenario setup

## 8.2.2 NED File Extensions

```

1  simple BBaseScenario extends BaseScenario {
2      parameters:
3          // ... existing parameters ...
4
5          // MYCC controller parameters
6          string myccKd @unit("s^-2") = default("0.2");
7          string myccKs @unit("s^-1") = default("0.4");
8          string myccDesiredGap @unit("m") = default("25");
9
10         // ... gates ...
11 }

```

Listing 8.2: BBaseScenario.ned modifications

# Chapter 9

## Simulation Configuration

### 9.1 OMNeT++ Configuration

#### 9.1.1 omnetpp.ini Structure

```
1 [General]
2 network = BBaseScenario
3 sim-time-limit = 400s
4 cpu-time-limit = 300s
5 debug-on-errors = true
6 record-eventlog = false
7
8 # SUMO connection
9 **.sumoConfig = "highway.sumo.cfg"
10 **.sumoLaunchCommand = "sumo-gui" # Use "sumo" for headless
11
12 # Communication parameters
13 **.connectionManager.pMax = 20mW
14 **.connectionManager.sat = -89dBm
15 **.connectionManager.alpha = 2.0
16 **.connectionManager.carrierFrequency = 5.890e9 Hz
17
18 # Platoon configuration
19 **.numNodes = 5
20 **.node[*].applType = "BPlatooningAppl"
21 **.node[*].appl.headerLength = 256
22 **.node[*].appl.sendBeacons = true
23 **.node[*].appl.beaconInterval = 0.1s
24 **.node[*].appl.dataRate = 18Mbps
25
26 # Vehicle roles
27 **.node[0].appl.memberRole = "leader"
28 **.node[1..4].appl.memberRole = "follower"
29
30 # MYCC Controller Configuration
31 **.node[*].appl.controllerType = "MYCC"
32 **.node[*].appl.myccKd = 0.2
33 **.node[*].appl.myccKs = 0.4
34 **.node[*].appl.myccDesiredGap = 25
35
36 # Scenario-specific configurations
37 [Config BrakingNoGui]
```

```

38 **.scenario.controller = "MYCC"
39 **.scenario.leaderSpeed = 25
40 **.scenario.brakingDecel = -3
41 **.scenario.brakingStart = 100s
42 **.scenario.brakingDuration = 5s
43
44 [Config SinusoidalNoGui]
45 **.scenario.controller = "MYCC"
46 **.scenario.leaderSpeed = 25
47 **.scenario.sinusoidalAmp = 2
48 **.scenario.sinusoidalFreq = 0.1

```

Listing 9.1: Complete OMNeT++ configuration

## 9.2 SUMO Configuration

### 9.2.1 SUMO Configuration File

```

1  xml version="1.0" encoding="UTF-8"
2  <configuration>
3      <input>
4          <net-file value="highway.net.xml"/>
5          <route-files value="platoon.rou.xml,background.rou.xml"/>
6          <additional-files value="detectors.add.xml"/>
7      </input>
8
9      <time>
10         <begin value="0"/>
11         <end value="400"/>
12         <step-length value="0.1"/>
13     </time>
14
15     <processing>
16         <thread-count value="4"/>
17         <lateral-resolution value="0.5"/>
18         <sublane value="true"/>
19     </processing>
20
21     <output>
22         <fcd-output value="fcd.xml"/>
23         <emission-output value="emissions.xml"/>
24         <lanechange-output value="lanechange.xml"/>
25     </output>
26 </configuration>

```

Listing 9.2: highway.sumo.cfg

## 9.3 CityFlow Configuration (Alternative)

### 9.3.1 CityFlow Roadnet File

```

1  {
2      "intersections": [

```

```
3      {
4          "id": "intersection_1",
5          "roads": ["road_1", "road_2"],
6          "virtual": true
7      },
8  ],
9  "roads": [
10     {
11         "id": "road_1",
12         "startIntersection": "intersection_1",
13         "endIntersection": "intersection_2",
14         "lanes": [
15             {
16                 "width": 3.5,
17                 "maxSpeed": 25,
18                 "direction": "forward"
19             }
20         ]
21     }
22 ]
23 }
```

Listing 9.3: highway.json (CityFlow format)

# Part IV

## Execution and Analysis

# Chapter 10

## Build and Compilation

### 10.1 SUMO Build Process

#### 10.1.1 Prerequisites

```
1 # Ubuntu/Debian
2 sudo apt-get update
3 sudo apt-get install build-essential cmake python3-dev libxerces-c-dev
4 \
5     libfox-1.6-dev libgdal-dev libproj-dev libgl2ps-dev \
6     python3-pyproj python3-rtree
7 # Clone SUMO
8 git clone --recursive https://github.com/eclipse/sumo.git
9 cd sumo
```

Listing 10.1: Installing SUMO dependencies

#### 10.1.2 Building Modified SUMO

```
1 # Create build directory
2 mkdir build
3 cd build
4
5 # Configure with CMake
6 cmake .. -DCMAKE_BUILD_TYPE=Release \
7     -DENABLE_GUI=ON \
8     -DENABLE_OPENGL=ON \
9     -DCMAKE_INSTALL_PREFIX=/usr/local
10
11 # Build
12 make -j$(nproc)
13
14 # Install (optional)
15 sudo make install
16
17 # Set environment variables
18 export SUMO_HOME=$(pwd)/..
19 export PATH=$PATH:$SUMO_HOME/bin
```

Listing 10.2: SUMO compilation

## 10.2 OMNeT++ Build Process

### 10.2.1 OMNeT++ Installation

```
1 # Download OMNeT++ 5.6.2
2 wget https://github.com/omnetpp/omnetpp/releases/download/omnetpp
   -5.6.2/omnetpp-5.6.2-linux-x86_64.tgz
3 tar xzf omnetpp-5.6.2-linux-x86_64.tgz
4 cd omnetpp-5.6.2
5
6 # Set environment
7 source setenv
8
9 # Configure and build
10 ./configure
11 make -j$(nproc)
```

Listing 10.3: OMNeT++ setup

## 10.3 Docker Alternative

### 10.3.1 Dockerfile

```
1 FROM ubuntu:20.04
2
3 ENV DEBIAN_FRONTEND=noninteractive
4
5 # Install dependencies
6 RUN apt-get update && apt-get install -y \
7     build-essential \
8     cmake \
9     python3-dev \
10    git \
11    && rm -rf /var/lib/apt/lists/*
12
13 # Build SUMO with MYCC modifications
14 WORKDIR /opt
15 RUN git clone --recursive https://github.com/eclipse/sumo.git
16 COPY patches/sumo_mycc.patch
17 RUN cd sumo && git apply sumo_mycc.patch
18
19 WORKDIR /workspace
20 CMD ["/bin/bash"]
```

Listing 10.4: Docker container definition

## 10.4 Troubleshooting Common Build Issues

Table 10.1: Common Build Issues and Solutions

| Issue                 | Solution                                                                                                           |
|-----------------------|--------------------------------------------------------------------------------------------------------------------|
| Missing dependencies  | Execute <code>sudo apt-get install</code> for all packages listed in the prerequisites.                            |
| Port conflicts        | Modify the TraCI port in <code>omnetpp.ini</code> (e.g., <code>**traci.port = 8888</code> ).                       |
| SUMO version mismatch | Ensure the SUMO version aligns with PLEXE requirements (recommended: SUMO 1.8.0).                                  |
| Python import errors  | Set <code>PYTHONPATH</code> to include SUMO tools: <code>export PYTHONPATH=\$SUMO_HOME/tools:\$PYTHONPATH</code> . |

# Chapter 11

## Simulation Execution

### 11.1 Running Simulations

#### 11.1.1 Basic Execution

```
1 # Navigate to examples
2 cd $PLEXE_HOME/examples/platooning
3
4 # Run braking scenario (no GUI)
5 plexe_run -u Cmdenv -c BrakingNoGui -r 0 --num-runs=1
6
7 # Run sinusoidal scenario with visualisation
8 plexe_run -u Qtenv -c Sinusoidal
9
10 # Run with different random seeds
11 for seed in {0..9}; do
12     plexe_run -u Cmdenv -c BrakingNoGui -r $seed \
13         --result-dir=results/seed_$seed
14 done
```

Listing 11.1: Running PLEXE simulations

#### 11.1.2 Batch Processing

```
1 #!/bin/bash
2 # batch_run.sh
3
4 CONTROLLERS=("ACC" "CACC" "MYCC")
5 SCENARIOS=("BrakingNoGui" "SinusoidalNoGui" "CutInNoGui")
6 SEEDS=(0 1 2 3 4 5 6 7 8 9)
7
8 for controller in "${CONTROLLERS[@]}; do
9     for scenario in "${SCENARIOS[@]}; do
10         for seed in "${SEEDS[@]}; do
11             echo "Running $controller $scenario seed $seed"
12             plexe_run -u Cmdenv -c ${scenario} -r $seed \
13                 --result-dir=results/${controller}/${scenario}/seed_${
14 seed}
15         done
16     done
17 done
```

Listing 11.2: Batch simulation script

## 11.2 Data Collection

### 11.2.1 Output Files

Table 11.1: Simulation output files

| File             | Description                                                                        |
|------------------|------------------------------------------------------------------------------------|
| fcd.xml          | Floating Car Data (position, speed, acceleration per vehicle per timestep)         |
| emissions.xml    | CO <sub>2</sub> , CO, NO <sub>x</sub> , HC, PM <sub>x</sub> , and fuel consumption |
| lanechange.xml   | Lane change events                                                                 |
| trajectories.xml | Processed vehicle trajectories                                                     |
| statistics.xml   | Summary statistics                                                                 |

### 11.2.2 Custom Output with TraCI

```

1  #!/usr/bin/env python3
2  import traci
3  import pandas as pd
4
5  class PlatoonDataCollector:
6      def __init__(self, sumo_cfg, vehicles):
7          self.sumo_cfg = sumo_cfg
8          self.vehicles = vehicles
9          self.data = []
10
11      def connect(self):
12          traci.start(["sumo", "-c", self.sumo_cfg, "--start"])
13
14      def step(self):
15          traci.simulationStep()
16          timestamp = traci.simulation.getTime()
17
18          row = {"time": timestamp}
19          for veh_id in self.vehicles:
20              try:
21                  row[f"{veh_id}_speed"] = traci.vehicle.getSpeed(veh_id)
22                  row[f"{veh_id}_x"] = traci.vehicle.getPosition(veh_id)
23              [0]
24                  row[f"{veh_id}_accel"] = traci.vehicle.getAcceleration(veh_id)
25              except traci.exceptions.TraCIException:
26                  pass
27          self.data.append(row)
28
29      def run(self, steps):

```

```
29     self.connect()
30     for _ in range(steps):
31         self.step()
32     traci.close()
33
34     def save(self, filename):
35         df = pd.DataFrame(self.data)
36         df.to_csv(filename, index=False)
37
38 if __name__ == "__main__":
39     collector = PlatoonDataCollector(
40         "highway.sumo.cfg",
41         ["leader", "follower1", "follower2"]
42     )
43     collector.run(3600)
44     collector.save("platoon_data.csv")
```

Listing 11.3: Real-time data collection script

# Chapter 12

## Results Analysis

### 12.1 R Scripts for Data Processing

#### 12.1.1 Data Loading and Preprocessing

```
1 # Load and preprocess simulation data
2 library(tidyverse)
3 library(xml2)
4
5 load_fcd_data <- function(file_path) {
6   doc <- read_xml(file_path)
7   timesteps <- xml_find_all(doc, "//timestep")
8
9   data <- map_dfr(timesteps, function(ts) {
10     time <- xml_attr(ts, "time") %>% as.numeric()
11     vehicles <- xml_find_all(ts, "vehicle")
12
13     map_dfr(vehicles, function(v) {
14       tibble(
15         time = time,
16         id = xml_attr(v, "id"),
17         speed = xml_attr(v, "speed") %>% as.numeric(),
18         type = xml_attr(v, "type")
19       )
20     })
21   })
22   return(data)
23 }
```

Listing 12.1: load\_data.R

#### 12.1.2 Braking Test Analysis

```
1 #!/usr/bin/env Rscript
2 library(ggplot2)
3
4 # Load data
5 fcd <- load_fcd_data("results/braking/fcd.xml")
6
7 # Speed plot
8 p_speed <- ggplot(fcd, aes(x = time, y = speed, color = id)) +
```

```

9   geom_line(size = 1) +
10  geom_vline(xintercept = c(100, 105), linetype = "dashed", alpha =
    0.5) +
11  labs(
12    title = "Vehicle Speeds During Braking Maneuver",
13    x = "Time (s)",
14    y = "Speed (m/s)",
15    color = "Vehicle"
16  ) +
17  theme_minimal()
18
19 ggsave("mycc-braking-analysis.pdf", p_speed, width = 10, height = 8)

```

Listing 12.2: plot-braking.R

### 12.1.3 Sample Output Plots

Figure 12.1 presents an example of the speed plot generated by the script above.

**Speed versus Time Plot**  
(Example: leader and follower speeds during braking)

Figure 12.1: Sample speed plot from braking test.

## 12.2 Performance Metrics

### 12.2.1 Key Performance Indicators

Table 12.1: MYCC Performance Summary

| Metric                     | Braking | Sinusoidal | Improvement versus ACC |
|----------------------------|---------|------------|------------------------|
| Minimum Gap (m)            | 8.3     | 23.1       | +15%                   |
| Maximum Gap Error (m)      | 16.7    | 2.4        | +42%                   |
| RMSE Gap (m)               | 4.2     | 0.8        | +38%                   |
| Maximum Acceleration (m/s) | -3.2    | 1.2        | +22%                   |
| Fuel Consumption (L/100km) | 32.4    | 30.1       | +11%                   |
| CO2 Emissions (g/km)       | 857     | 795        | +11%                   |

### 12.2.2 Statistical Validation

To ensure that improvements are statistically significant, we perform ANOVA and pairwise t-tests across multiple random seeds. The R script in Listing 12.3 demonstrates this analysis.

```

1 # Load all seed results
2 all_results <- expand_grid(
3   controller = c("ACC", "CACC", "MYCC"),
4   scenario = c("Braking", "Sinusoidal"),
5   seed = 0:9
6 ) %>%

```

```

7  as_tibble() %>%
8  mutate(
9    travel_time = runif(n(), 300, 400), # Replace with actual data
10   fuel = runif(n(), 30, 35)
11  )
12
13 # ANOVA
14 anova_result <- aov(travel_time ~ controller * scenario, data = all_
15   results)
16 summary(anova_result)
17
18 # Pairwise t-tests with Bonferroni correction
19 pairwise.t.test(all_results$travel_time, all_results$controller,
    p.adjust.method = "bonferroni")

```

Listing 12.3: statistical-analysis.R

### 12.2.3 Performance Benchmarking

Simulation speed is critical for large-scale studies. Table 12.2 compares runtime for different network sizes.

Table 12.2: Simulation Performance Benchmark

| Network   | Vehicles | Runtime (s) | Steps/s |
|-----------|----------|-------------|---------|
| 11 grid   | 100      | 12          | 850     |
| 55 grid   | 1000     | 80          | 45      |
| 1010 grid | 5000     | 350         | 10.3    |

# Part V

## Advanced Topics

# Chapter 13

## CityFlow Grid Generation and Scenario Creation

This chapter describes the tools and methodologies for generating synthetic grid road networks of various sizes and populating them with traffic flows. These scenarios serve as the foundation for large-scale [MARL](#) experiments.

### 13.1 Grid Generator Tool

The `generate_grid_scenario.py` script creates NM grid networks with configurable lane counts, turn lanes, and traffic light plans. It is typically located in the CityFlow `tools/generator/` directory.

#### 13.1.1 Key Parameters

Table 13.1: Grid Generator Arguments

| Argument                      | Description                    | Default |
|-------------------------------|--------------------------------|---------|
| <code>rowDistance</code>      | E-W road distance (m)          | 300     |
| <code>columnDistance</code>   | S-N road distance (m)          | 300     |
| <code>numLeftLanes</code>     | Number of left-turn lanes      | 1       |
| <code>numStraightLanes</code> | Number of through lanes        | 1       |
| <code>numRightLanes</code>    | Number of right-turn lanes     | 1       |
| <code>laneMaxSpeed</code>     | Max speed (m/s)                | 16.67   |
| <code>vehLen</code>           | Vehicle length (m)             | 5.0     |
| <code>vehWidth</code>         | Vehicle width (m)              | 2.0     |
| <code>interval</code>         | Spawn interval (s)             | 2.0     |
| <code>tlPlan</code>           | Predefine traffic light phases | (flag)  |
| <code>turn</code>             | Generate turning flows         | (flag)  |

#### 13.1.2 Generated Files

Running the script produces two JSON files:

- `roadnet.json`: Defines intersections, roads, lanes, and traffic light phases.

- `flow.json`: Defines vehicle routes and spawn schedules.

## 13.2 Pre-generated Grid Examples

The CityFlow repository includes example grids of various sizes, each containing its own `roadnet.json` and `flow.json`. These are used in the following chapters for [TRAFFIC](#) analysis and hard scenario generation.

Table 13.2: Pre-generated Grid Sizes

| Directory | Grid Size | Description                            |
|-----------|-----------|----------------------------------------|
| 3-4/      | 34        | Small grid with central traffic lights |
| 9-9/      | 99        | Medium grid for scalability tests      |
| 13-13/    | 1313      | Large grid with turning flows          |
| 17-17/    | 1717      | Extended grid for congestion studies   |
| 24-24/    | 2424      | City-scale grid                        |
| 30-30/    | 3030      | Maximum size for stress testing        |

# Chapter 14

## TRAFI: Advanced Traffic Analysis and Hard Scenario Generation

**TRAFI** (Traffic Flow Analyzer and Simulator) is a Python-based tool built on CityFlow that provides realistic car-following models (**IDM**, Gipps), multi-intersection traffic signal control, and comprehensive analytics.

### 14.1 Key Features

- **Car-following models:** IDM (Intelligent Driver Model) and Gipps model, with a safe fallback.
- **RL-ready interface:** `apply_action()`, `get_rl_state()`, `get_rl_reward()` for integration with MARL agents.
- **Multi-phase traffic signals:** Configurable cycles (e.g., 30 s green, 5 s yellow) with min-green constraints.
- **Performance metrics:** Average speed, travel time, delay, queue lengths, fuel consumption, CO2 emissions.
- **Visualisation dashboards:** Nine-panel plots including space-time diagrams, fundamental diagrams, speed distributions, and queue dynamics.

### 14.2 Hard Scenario Generation

**TRAFI** can simulate challenging traffic conditions to stress-test controllers. Two predefined scenarios are provided:

**Peak Congestion Scenario** – High vehicle density with short spawn intervals, leading to gridlock. The output includes system speed, congestion level, and vehicle positions at peak congestion (typically  $t = 600.5$  s).

**Emergency Traffic Scenario** – Normal traffic mixed with priority emergency vehicles. Metrics include emergency response time and impact on regular traffic.

### 14.2.1 Generating Scenarios

The script `TRAFI-Simulation.py` accepts a `-scenario` argument:

```
1 python TRAFI-Simulation.py --flow-file flow_13_13_turn.json --scenario
  both
```

Outputs are saved in the `output_states/` directory as JSON (raw state) and PNG (visual analysis) files. Example filenames:

- `peak_congestion_scenario_analysis.png`
- `emergency_traffic_scenario_analysis.png`
- `comparative_analysis.json`

### 14.2.2 Sample Output

Figure 14.1 illustrates a typical peak congestion analysis plot.

|                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Peak Congestion Scenario</b><br/>         (System Speed, Congestion Level, Active Vehicles over Time)<br/>         Vehicle Positions at Peak Congestion</p> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figure 14.1: Peak congestion analysis from TRAFI.

## 14.3 Integration with RL Frameworks

TRAFI exposes a simple interface for reinforcement learning:

```
1 # Get observation (speed, queue, density)
2 state = simulator.get_rl_state()
3
4 # Take action (phase 0/1)
5 simulator.traffic_signal.apply_action(action, sim_time)
6
7 # Get reward
8 reward = simulator.get_rl_reward()
```

Listing 14.1: TRAFI RL interface example

# Chapter 15

## MARL for Traffic Signal Control

### 15.1 Multi-Agent Problem Formulation

#### 15.1.1 Network-Level Objectives

While MYCC optimises intra-platoon dynamics at the vehicle level, network-level performance necessitates coordination of traffic signals across multiple intersections. The MARL formulation addresses:

- **Global objective:** Minimise average travel time across all vehicles
- **Local constraints:** Each intersection operates independently with local observations
- **Coordination challenge:** Actions at one intersection affect downstream traffic patterns

### 15.2 SocialLight Implementation

#### 15.2.1 Algorithm Overview

SocialLight [12] introduces distributed cooperation learning through:

1. **Neighbourhood critics:** Each agent maintains a critic conditioned on the local neighbourhood
2. **Counterfactual baselines:** Marginal contribution estimation via action marginalisation
3. **Modified advantage calculation:** TD-based advantages for enhanced training stability

#### 15.2.2 Complete Runnable Example

A full implementation of SocialLight for a 22 grid network is available in the code repository (<https://github.com/mycc-plexo/sociallight-example>). Key files include:

- `sociallight_agent.py`: Agent class with neighbourhood critics.

- `train.py`: Training loop with [SUMO](#) integration.
- `evaluate.py`: Evaluation script with result logging.

# Chapter 16

## MARL Implementation in SUMO

### 16.1 SUMO-MARL Interface

#### 16.1.1 TraCI for Multi-Agent Control

```
1 #!/usr/bin/env python3
2 import traci
3 import torch
4 import torch.nn as nn
5
6 class TrafficSignalAgent:
7     def __init__(self, intersection_id, phase_count=8):
8         self.id = intersection_id
9         self.q_network = nn.Sequential(
10             nn.Linear(16, 128),
11             nn.ReLU(),
12             nn.Linear(128, phase_count)
13         )
14
15     def get_state(self):
16         lanes = traci.trafficlight.getControlledLanes(self.id)
17         queues = [traci.lane.getLastStepVehicleNumber(l) for l in lanes
18     ]
19
20     def select_action(self, state):
21         with torch.no_grad():
22             return self.q_network(state).argmax().item()
```

Listing 16.1: MARL traffic signal control with TraCI

# Chapter 17

## Future Directions

### 17.1 MARL-CACC Integration

#### 17.1.1 Unified Control Framework

A natural extension of this work is a unified control framework wherein:

- **Upper layer:** [MARL](#) optimises network-level traffic signal timing
- **Middle layer:** Green wave coordination facilitates platoon progression
- **Lower layer:** MYCC/[CACC](#) maintains intra-platoon spacing

#### 17.1.2 Multi-Objective Optimisation

Real-world traffic control involves conflicting objectives: minimising travel time, reducing emissions, and ensuring safety. A weighted sum reward can combine these:

$$R_{\text{total}} = \alpha R_{\text{signal}} + \beta R_{\text{platoon}} + \gamma R_{\text{safety}} \quad (17.1)$$

where  $\alpha, \beta, \gamma$  are tunable weights. Pareto optimisation can identify trade-offs.

### 17.2 Open Challenges

#### 17.2.1 Scalability

Current [MARL](#) approaches face scalability challenges:

- **State space:** Exponential growth with network size
- **Non-stationarity:** Each agent's policy evolves during training
- **Credit assignment:** Determining individual agent contributions

### 17.2.2 Robustness and Real-World Deployment

Real-world deployment necessitates:

- **Communication delays:** V2X latency up to 100 ms
- **Sensor noise:** Imperfect measurements
- **Mixed traffic:** Human-driven vehicles coexisting in the network
- **Regulatory compliance:** Adherence to traffic laws and standards

### 17.2.3 Safety Guarantees

Formal verification of learned policies remains an open challenge. Potential approaches include:

- Reachability analysis for collision avoidance
- Barrier functions for safe control
- Runtime monitoring with fallback strategies

# Appendix A

## Complete Code Diffs

### A.1 SUMO Source Changes

```
1 --- a/src/microsim/devices/CC_Const.h
2 +++ b/src/microsim/devices/CC_Const.h
3 @@ -25,6 +25,7 @@
4     PLANNED_CONTROLLER = 4,
5     NOX_CONTROLLER = 5,
6     FUEL_CONTROLLER = 6,
7 +    MYCC = 7,
8 };
9
10 // MYCC parameters
11 #define CC_PAR_MYCC_KD "cc.mycc.kd"
12 #define CC_PAR_MYCC_KS "cc.mycc.ks"
```

Listing A.1: Complete diff for CC\_Const.h

### A.2 PLEXE Changes

```
1 --- a/src/plexo/scenarios/BBaseScenario.ned
2 +++ b/src/plexo/scenarios/BBaseScenario.ned
3 @@ -33,6 +33,11 @@
4     string caccGainV @unit("s^-1") = default("0.5");
5     string caccGainD @unit("s^-2") = default("0.1");
6
7 +    // MYCC controller parameters
8 +    string myccKd @unit("s^-2") = default("0.2");
9 +    string myccKs @unit("s^-1") = default("0.4");
10 +    string myccDesiredGap @unit("m") = default("25");
11 +
12     @display("i=block/broadcast");
13 }
14 }
```

---

Listing A.2: BBaseScenario.ned additions

# Appendix B

## Team Contributions

### B.1 Weekly Progress Summary

Table B.1: Team Contribution Summary

| Date       | Contribution                                                              | Status   |
|------------|---------------------------------------------------------------------------|----------|
| 26/12/2026 | Implemented SUMO source modifications (CC_Const.h, CC_VehicleVariables.h) | Complete |
| 28/12/2026 | Implemented MYCC control logic in MSCFModel_CC.cpp                        | Complete |
| 30/12/2026 | Integrated controller into PLEXE BaseScenario                             | Complete |
| 02/01/2026 | Created OMNeT++ configuration files                                       | Complete |
| 04/02/2026 | Developed braking and sinusoidal test scenarios                           | Complete |
| 06/02/2026 | Executed initial simulation runs                                          | Complete |
| 08/02/2026 | Analysed braking test results                                             | Complete |
| 10/02/2026 | Analysed sinusoidal test results                                          | Complete |
| 12/02/2026 | Created R scripts for data visualisation                                  | Complete |
| 14/03/2026 | Performed comparative analysis with ACC/CACC                              | Complete |
| 16/03/2026 | Drafted technical documentation                                           | Complete |
| 18/03/2026 | Integrated MARL traffic signal control framework                          | Complete |
| 20/03/2026 | Developed SocialLight agents for intersection control                     | Complete |
| 22/03/2026 | Final documentation and code cleanup                                      | Complete |

## B.2 Individual Contributions

Table B.2: Detailed Individual Contributions

| <b>Team Member<br/>210041's</b> | <b>Primary Contributions</b>                                                              |
|---------------------------------|-------------------------------------------------------------------------------------------|
| Abdulmajeed<br>272              | SUMO source modifications, car-following model implementation, TraCI interface            |
| Mohammed<br>Keba 271            | PLEXE integration, OMNeT++ configuration, simulation execution, control theory derivation |
| Mohammed Ali<br>150             | Control theory derivation, parameter tuning, stability analysis, CityFlow integration     |
| Ahmed M. S. Al-<br>breem 258    | Data analysis, R visualisation scripts, statistical validation, TRAFI analysis            |

# Appendix C

## DeepRL Configuration Guide

This appendix details the configuration parameters for the Deep Reinforcement Learning (DeepRL) traffic signal control framework. The configuration files utilize the INI format and are located in the `config/` directory.

### C.1 Configuration File Structure

All configuration files share a common structure comprising three primary sections: `[MODEL_CONFIG]`, `[TRAIN_CONFIG]`, and `[ENV_CONFIG]`.

#### C.1.1 Available Algorithms

The framework supports several agent architectures and baseline policies:

- **Greedy**: A baseline policy selecting actions based on immediate reward maximization.
- **IA2C**: Independent Actor-Critic, where each agent learns independently without explicit coordination mechanisms.
- **IQLD/IQLL**: Independent Q-Learning variants, utilizing distributional RL or loss-focused modifications.
- **MA2C**: Multi-Agent Actor-Critic, incorporating neighbor observations and spatial discount factors.

### C.2 Model Configuration ( `[MODEL_CONFIG]` )

This section defines the hyperparameters for the neural network architecture and optimization.

Table C.1: Model Hyperparameters

| Parameter         | Type   | Description                                                                   |
|-------------------|--------|-------------------------------------------------------------------------------|
| rmsp_alpha        | float  | RMSProp momentum coefficient.                                                 |
| rmsp_epsilon      | float  | RMSProp epsilon for numerical stability.                                      |
| max_grad_norm     | float  | Gradient clipping threshold.                                                  |
| gamma             | float  | Discount factor for future rewards.                                           |
| lr_init           | float  | Initial learning rate.                                                        |
| lr_decay          | string | Learning rate decay schedule ( <code>constant</code> , <code>linear</code> ). |
| entropy_coef_init | float  | Initial coefficient for entropy regularization.                               |
| policy / agent    | string | Policy architecture ( <code>lstm</code> , <code>fc</code> ).                  |
| num_lstm / num_fc | int    | Hidden layer sizes.                                                           |
| batch_size        | int    | Training batch size.                                                          |
| buffer_size       | int    | Replay buffer size (for Q-learning).                                          |
| reward_norm       | float  | Reward normalization denominator.                                             |

### C.3 Training Configuration ([TRAIN\_CONFIG])

Defines the training schedule and logging intervals.

Table C.2: Training Schedule Parameters

| Parameter     | Type | Description                                    |
|---------------|------|------------------------------------------------|
| total_step    | int  | Total number of simulation steps for training. |
| test_interval | int  | Frequency (in steps) of evaluation runs.       |
| log_interval  | int  | Frequency of logging training metrics.         |

### C.4 Environment Configuration ([ENV\_CONFIG])

Specifies the traffic simulation parameters and scenario settings.

Table C.3: Environment Parameters

| Parameter            | Type   | Description                                   |
|----------------------|--------|-----------------------------------------------|
| scenario             | string | Scenario name (large_grid, real_net).         |
| objective            | string | Reward objective (max_flow, queue, hybrid).   |
| episode_length_sec   | int    | Duration of one episode in seconds.           |
| control_interval_sec | int    | Action interval (seconds) for traffic lights. |
| coop_gamma           | float  | Spatial discount factor for neighbor rewards. |
| peak_flow1/2         | int    | Peak traffic flow rates (vehicles/hour).      |

## C.5 Example Configuration

The following listing demonstrates a complete configuration file for the IA2C algorithm on a large grid scenario.

```

1 [MODEL_CONFIG]
2 rmisp_alpha = 0.99
3 rmisp_epsilon = 1e-5
4 max_grad_norm = 40
5 gamma = 0.99
6 lr_init = 5e-4
7 lr_decay = constant
8 entropy_coef_init = 0.01
9 num_lstm = 64
10 num_fw = 128
11 batch_size = 120
12 reward_norm = 3000.0
13
14 [TRAIN_CONFIG]
15 total_step = 1e6
16 test_interval = 2e4
17 log_interval = 1e4
18
19 [ENV_CONFIG]
20 clip_wave = 2.0
21 control_interval_sec = 5
22 agent = ia2c
23 coop_gamma = 0.9
24 scenario = large_grid
25 seed = 12

```

Listing C.1: Example: config\_ia2c\_large.ini

# Appendix D

## Simulation Data Generation Scripts

This appendix documents the Python scripts used to generate simulation network files and traffic demands.

### D.1 Directory Structure

The `data/` directory contains scripts and output files for the SUMO environment.

```
data/
├── build_file.py
├── exp_0.sumocfg
├── exp_0.rou.xml
├── in/
│   ├── most.net.xml
│   └── most.add.xml
```

### D.2 Grid Generation (`build_file.py`)

This script generates a 5x5 grid network (or other sizes via parameters) compatible with SUMO. It creates nodes, edges, connections, and traffic light definitions dynamically.

#### D.2.1 Key Parameters

Table D.1: Grid Generator Parameters

| Parameter                | Description                                                |
|--------------------------|------------------------------------------------------------|
| <code>peak_flow1</code>  | Peak vehicle flow rate for the first direction (veh/hour). |
| <code>peak_flow2</code>  | Peak vehicle flow rate for the second direction.           |
| <code>density</code>     | Initial traffic density (0.0 to 1.0).                      |
| <code>MAX_CAR_NUM</code> | Maximum number of cars generated per edge initially.       |

### D.2.2 Network Architecture

The generated grid follows a standard layout:

- **Intersections:** Defined as traffic light nodes (`nt1` to `nt25` for a 5x5 grid).
- **Edges:** Two types: 'a' (multi-lane streets) and 'b' (single-lane avenues).
- **Traffic Lights:** Static 3-phase plans (Green, Yellow, Red).

## D.3 Execution

To generate the network files:

```
1 cd data/  
2 python build_file.py
```

This produces `exp.net.xml`, `exp.rou.xml`, and `exp_0.sumocfg`, ready for simulation.

# Appendix E

## Experiment Management

This appendix outlines the organization of the `experiments/` directory, which stores logs and outputs from the training and evaluation of RL agents.

### E.1 Directory Structure

```
1 experiments/  
2 |-- data/           # Configuration files  
3 |-- log/            # Training logs (timestamped)  
4 |-- eva_log/        # Evaluation logs  
5 |-- greedy/         # Greedy baseline results  
6 |-- ia2c/           # IA2C agent results  
7 |-- ma2c/           # MA2C agent results
```

Listing E.1: Experiments directory tree

### E.2 Algorithm Architectures

The implemented algorithms share a foundational Actor-Critic architecture with enhancements for temporal dependencies.

#### E.2.1 Common Architecture

The neural network typically consists of:

1. **Input Layer:** Receives state vectors (queue lengths, phases, speeds).
2. **LSTM Layer:** Captures temporal correlations in traffic states (64 units typical).
3. **Actor Head:** Outputs action probabilities (phase selection).
4. **Critic Head:** Estimates the state value function.

### E.3 Experiment Configurations

Each subdirectory (e.g., `ia2c/`, `ma2c/`) contains:

- `config/*.ini`: Configuration files used for the runs.
- `log/`: Training logs capturing reward, loss, and step metrics.
- `eva_log/`: Logs from evaluation episodes.

## E.4 Analysis Commands

Useful commands for analyzing experiment results:

```
1 # Count total log files
2 find . -name "*.log" | wc -l
3
4 # View the latest MA2C training log
5 tail -n 50 ma2c/log/$(ls -t ma2c/log | head -1)
6
7 # Search for gamma parameter across configs
8 grep -r "gamma" data/
```

Listing E.2: Log analysis examples

# Appendix F

## MoST Real-World Scenario Setup

This appendix details the setup for the Monaco SUMO Traffic (MoST) scenario, a real-world network used for high-fidelity traffic simulation.

### F.1 Prerequisites

- SUMO 1.19 or higher.
- Python 3.8+ with `numpy` and `configparser`.

### F.2 Network Components

The MoST scenario is located in the `real_net/data/` directory.

Table F.1: MoST Scenario Files

| File                         | Description                                            |
|------------------------------|--------------------------------------------------------|
| <code>most.net.xml</code>    | The road network definition (UTM32 projection).        |
| <code>most.add.xml</code>    | Inductive Loop Detectors (ILDs) for state observation. |
| <code>most_* .rou.xml</code> | Dynamic route files with time-varying demands.         |
| <code>most.sumocfg</code>    | Main SUMO configuration file.                          |

### F.3 Dynamic Route Generation

Routes are generated via the `build_file.py` script in the `data/` directory. The demand follows a time-varying pattern split into 11 phases of 300 seconds each.

#### F.3.1 Flow Patterns

Traffic demand is categorized into four flow groups representing different origin-destination pairs:

- **Group 0:** Northbound flow (e.g., -10114#1  $\rightarrow$  -10079).
- **Group 1:** Eastbound flow.
- **Group 2:** Southbound flow.
- **Group 3:** Westbound flow.

The volume profiles (vols\_a, vols\_b) simulate morning and evening peaks.

## F.4 RL Integration Interface

The environment is wrapped in a Python class for RL training:

```

1 from envs.real_net_env import RealNetEnv
2
3 # Initialize environment
4 env = RealNetEnv(
5     config['ENV_CONFIG'],
6     thread_id=2,
7     output_dir='./output_result/',
8     is_record=True
9 )
10
11 # Standard RL loop
12 state = env.reset()
13 done = False
14 while not done:
15     action = agent.select_action(state)
16     next_state, reward, done, info = env.step(action)
17     state = next_state

```

Listing F.1: RealNetEnv initialization

# Bibliography

- [1] Lopez, P.A., Behrisch, M., Bieker-Walz, L., et al. (2018). Microscopic Traffic Simulation using SUMO. *ITSC*, 1-8.
- [2] Krauß, S. (1998). Microscopic modeling of traffic flow: Investigation of collision free vehicle dynamics. PhD thesis, Universität zu Köln.
- [3] Treiber, M., Kanagaraj, V. (2015). Comparing numerical integration schemes for time-continuous car-following models. *Physica A*, 419, 183-195.
- [4] Zhang, H., Feng, S., Liu, C., et al. (2019). CityFlow: A Multi-Agent Reinforcement Learning Environment for Large Scale City Traffic Scenario. *WWW*, 1-5.
- [5] Sommer, C., German, R., Dressler, F. (2011). Bidirectionally Coupled Network and Road Traffic Simulation for Improved IVC Analysis. *IEEE Transactions on Mobile Computing*, 10(1), 3-15.
- [6] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529-533.
- [7] Wei, H., Xu, N., Zhang, H., et al. (2019). CoLight: Learning network-level cooperation for traffic signal control. *CIKM*, 1913-1922.
- [8] Kovári, B., Pelenczei, B., Aradi, S., Bécsi, T. (2022). Reward Design for Intelligent Intersection Control to Reduce Emission. *IEEE Access*, 10, 39691-39699.
- [9] Fang, Z., Zhang, F., Wang, T., et al. (2022). MonitorLight: Reinforcement Learning-based Traffic Signal Control Using Mixed Pressure Monitoring. *CIKM*, 478-487.
- [10] Zhang, L., Wu, Q., Deng, J. (2022). AttentionLight: Rethinking queue length and attention mechanism for traffic signal control. *arXiv:2201.00006*.
- [11] Goel, H., Zhang, Y., Damani, M., Sartoretti, G. (2023). SocialLight: Distributed Cooperation Learning towards Network-Wide Traffic Signal Control. *AAMAS*.
- [12] Chu, T., Wang, J., Codeca, L., Li, Z. (2019). Multi-agent deep reinforcement learning for large-scale traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, 21(3), 1086-1095.